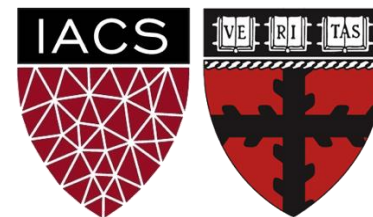
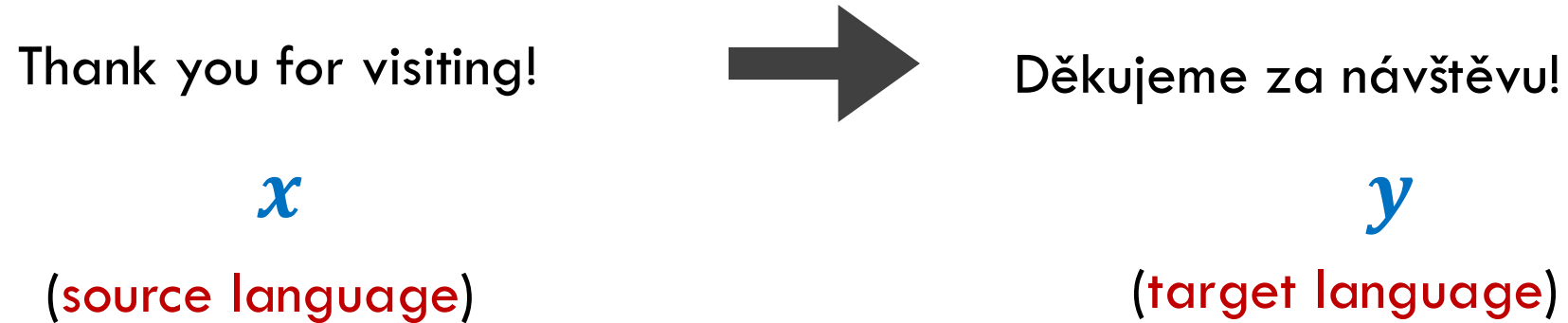


Machine Translation

And the power of Attention



Machine Translation (MT) is an NLP task that aims to convert text from one language to another.



Machine Translation (MT) is an NLP task that aims to convert text from one language to another.

9th century: Al-Kindi (cryptographer)

17th century: René Descartes theorized about a universal, symbolic language **1946:**

Warren Weaver had a seminal publication

1950s: First huge efforts; MIT, IBM, US Government. Motivated by the Cold War.

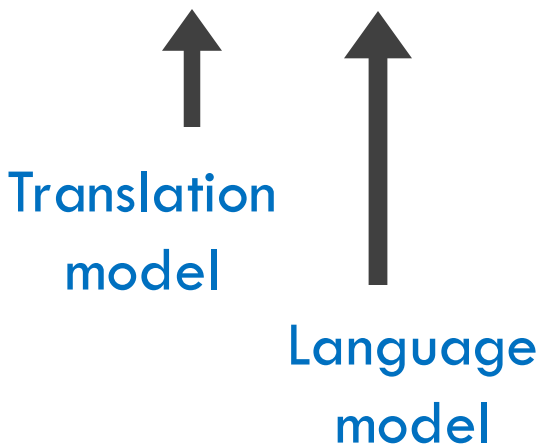
1990s – 2014: Statistical MT.

2014 – present: Neural MT (Deep Learning)

1990s – 2014: Statistical MT

$$\begin{aligned} & \operatorname{argmax}_y P(y|x) \\ &= \operatorname{argmax}_y P(x|y)P(y) \end{aligned}$$

1990s – 2014: Statistical MT

$$\begin{aligned} & \operatorname{argmax}_y P(y|x) \\ &= \operatorname{argmax}_y P(x|y) P(y) \end{aligned}$$


Translation
model

Language
model

1990s – 2014: Statistical MT

$$\operatorname{argmax}_y P(x|y)P(y)$$

We estimate $P(x|y)$ from a parallel corpus.

1990s – 2014: Statistical MT

$$\operatorname{argmax}_y P(x|y)P(y)$$

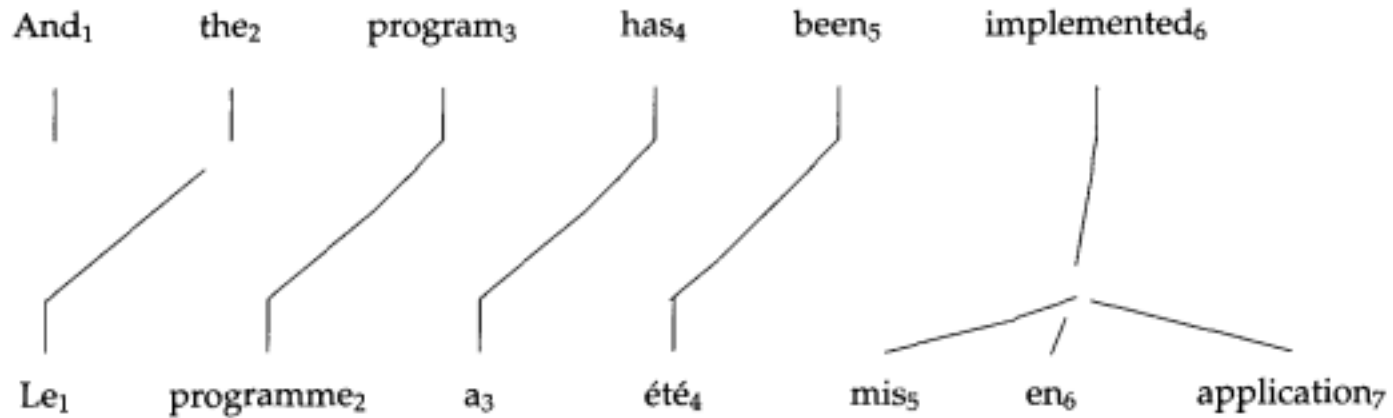
We estimate $P(x|y)$ from a parallel corpus.

Technically, we're actually interested in:

$$\operatorname{argmax}_y P(x, a|y)P(y)$$

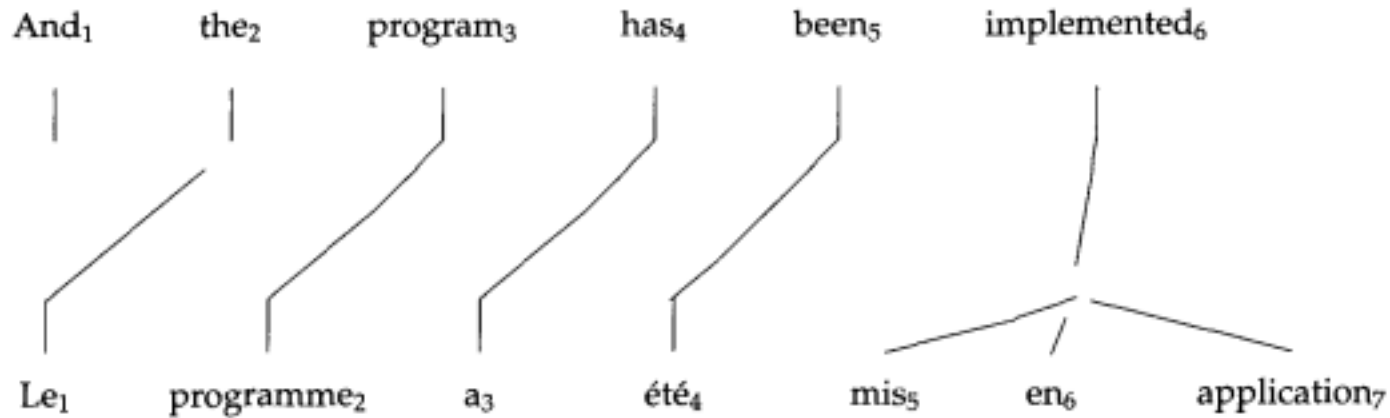

Def: **alignment** is the correspondence between particular words in different languages

1990s – 2014: Statistical MT



Some 1-to-0 mappings, 1-to-1 mappings, 1-to-many mappings, etc

1990s – 2014: Statistical MT



Alignment is complex and can be based on:

- particular words aligning
- sentence positions
- word “fertility” (the # of words a word spans to)
- much more

1990s – 2014: Statistical MT

$$\operatorname{argmax}_y P(x|y)P(y)$$

How in the world can we compute this **argmax**?! Exploding/infinite y 's!

1990s – 2014: Statistical MT

$$\operatorname{argmax}_y P(x|y)P(y)$$

How in the world can we compute this **argmax**?! Exploding/infinite y 's!

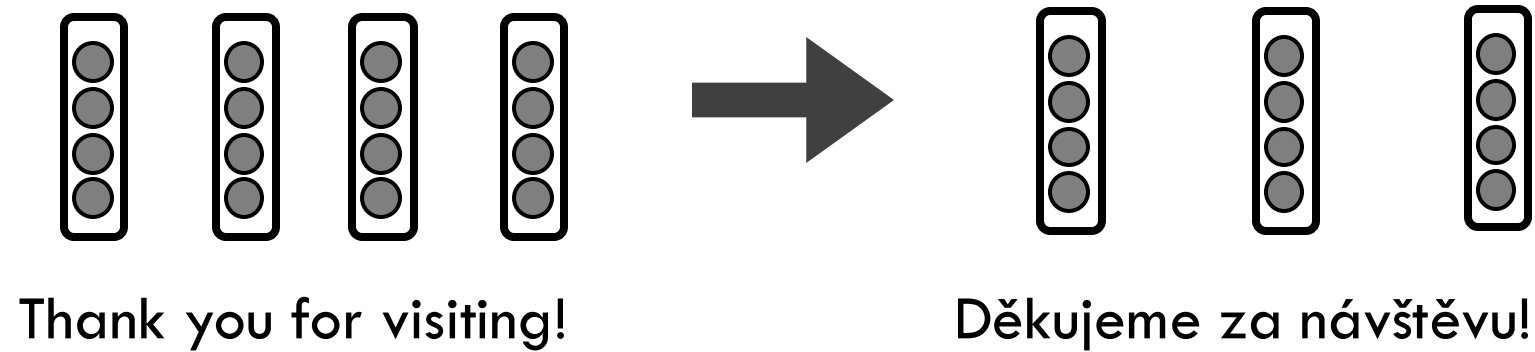
Neural MT (deep learning approach) needs to address this, too.

1990s – 2014: Statistical MT

SUMMARY

- SMT was largely successful and useful
- SOTA systems were incredibly complex (e.g., [Moses](#) and [GIZA++](#))
- Ridiculous amounts of manual feature engineering
- Relied on external resources like phrase translation tables

We want to produce a **variable-length** output
(e.g., **n** $\rightarrow$ **m** predictions)



Sequence-to-Sequence (seq2seq)

- If our input is a sentence in **Language A**, and we wish to translate it to **Language B**, it is clearly sub-optimal to translate word by word (like our current models are suited to do).
- Instead, let a ***sequence*** of tokens be the unit that we ultimately wish to work with (a sequence of length **N** may emit a sequences of length **M**)
- **seq2seq** models are comprised of **2 RNNs**: 1 encoder, 1 decoder

Types of Conditional Prediction

Many-to-1 classification

$$P(y|X)$$

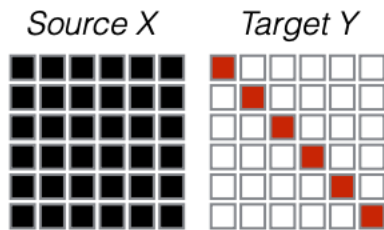


Many-to-many classification

Non-autoregressive Conditioned Prediction

$$P(Y|X) = \prod_{i=1}^{|Y|} P(y_i|X)$$

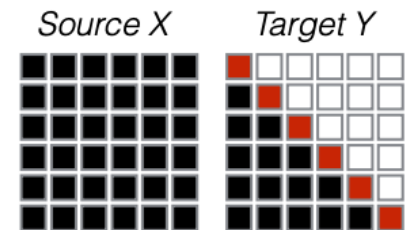
(e.g. sequence labeling, non-autoregressive MT)



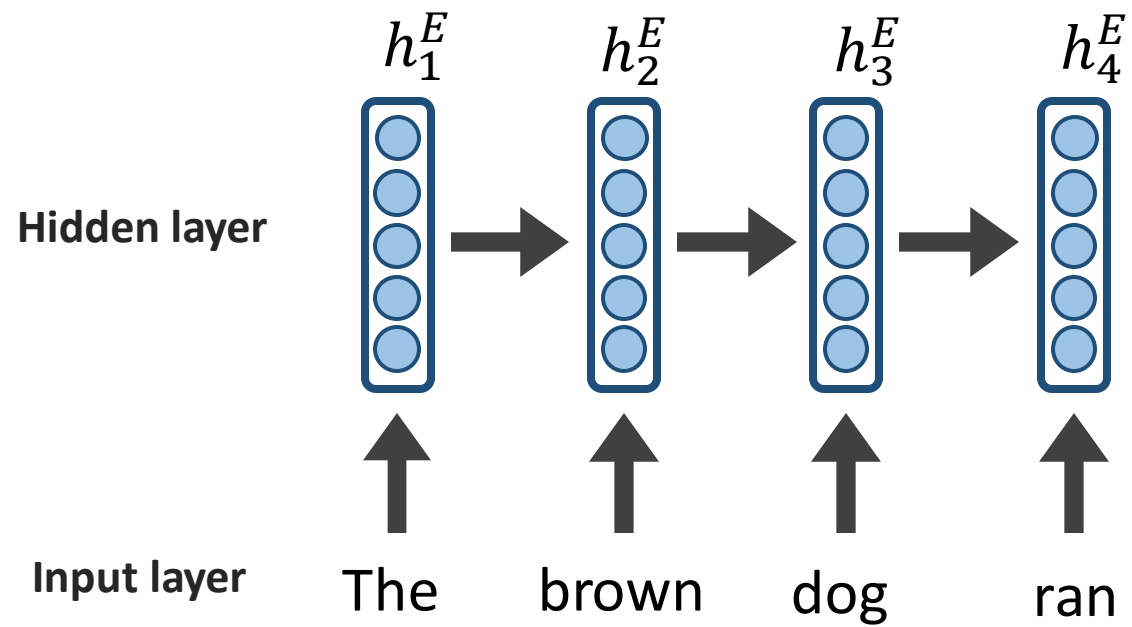
Autoregressive Conditioned Prediction

$$P(Y|X) = \prod_{i=1}^{|Y|} P(y_i|X, y_1, \dots, y_{i-1})$$

(e.g. seq2seq model)



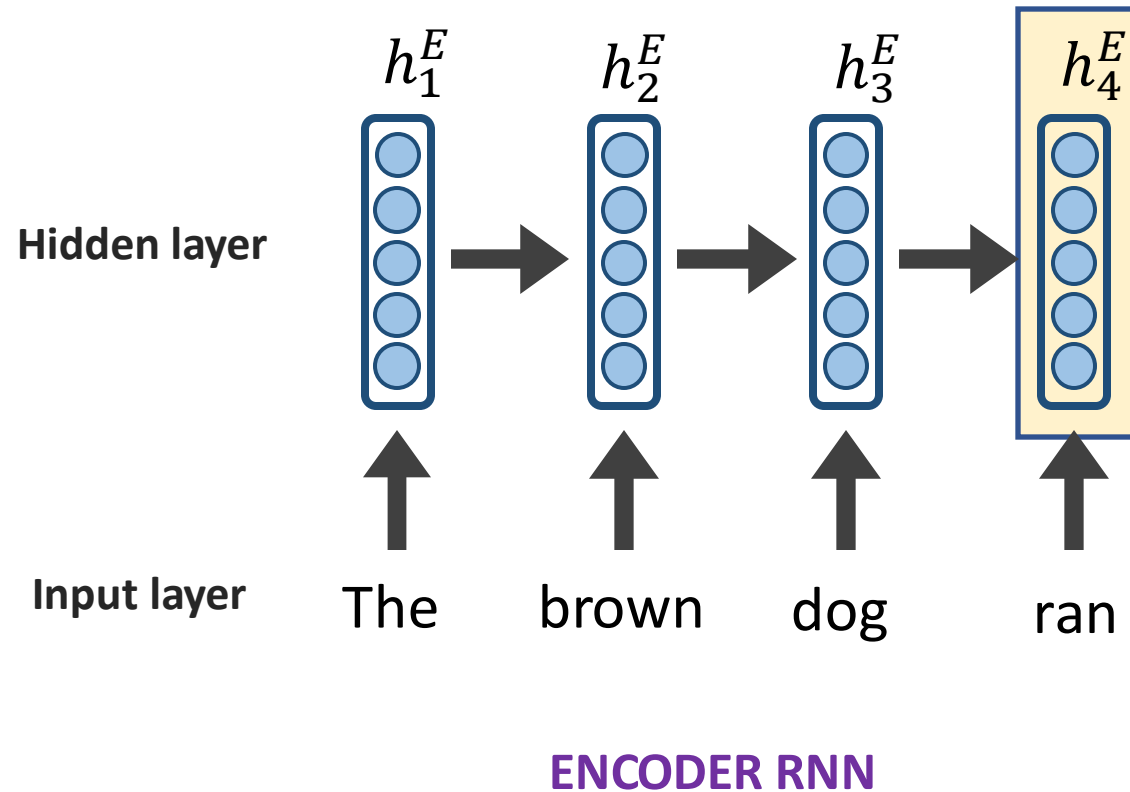
Sequence-to-Sequence (seq2seq)



ENCODER RNN

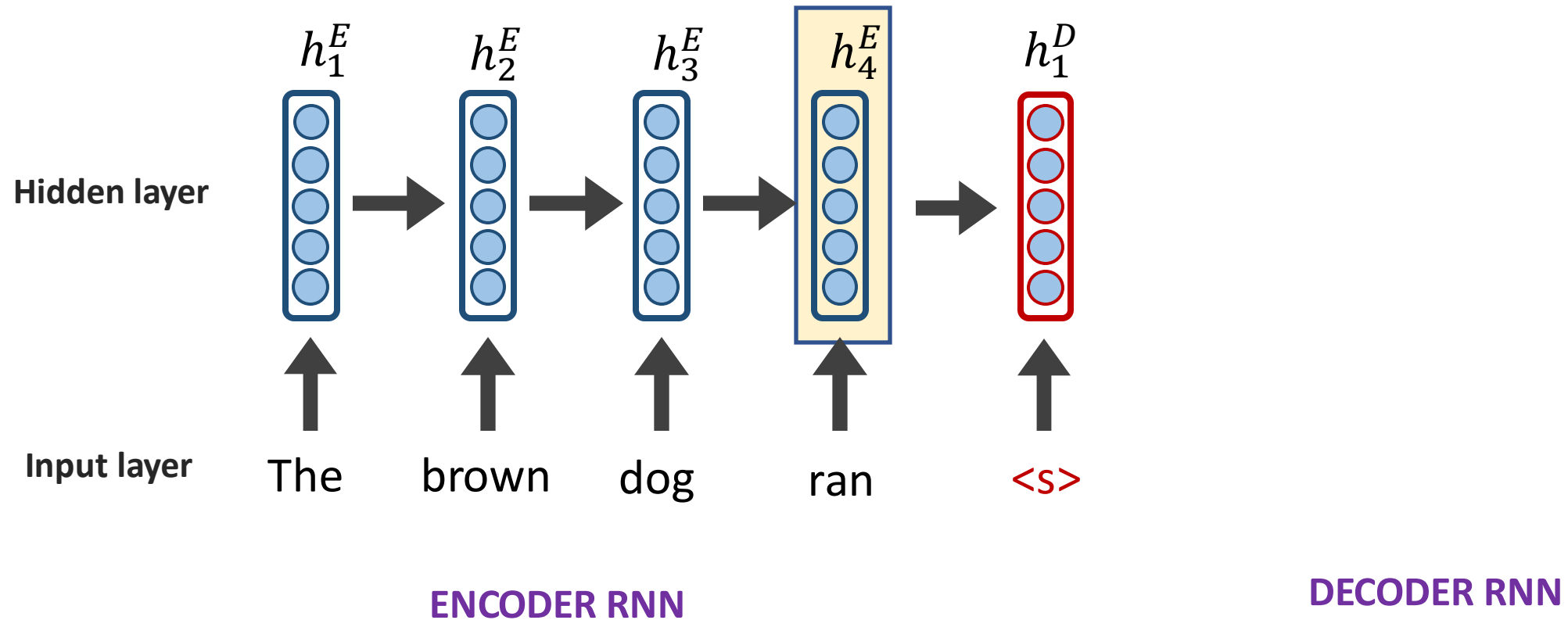
Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN



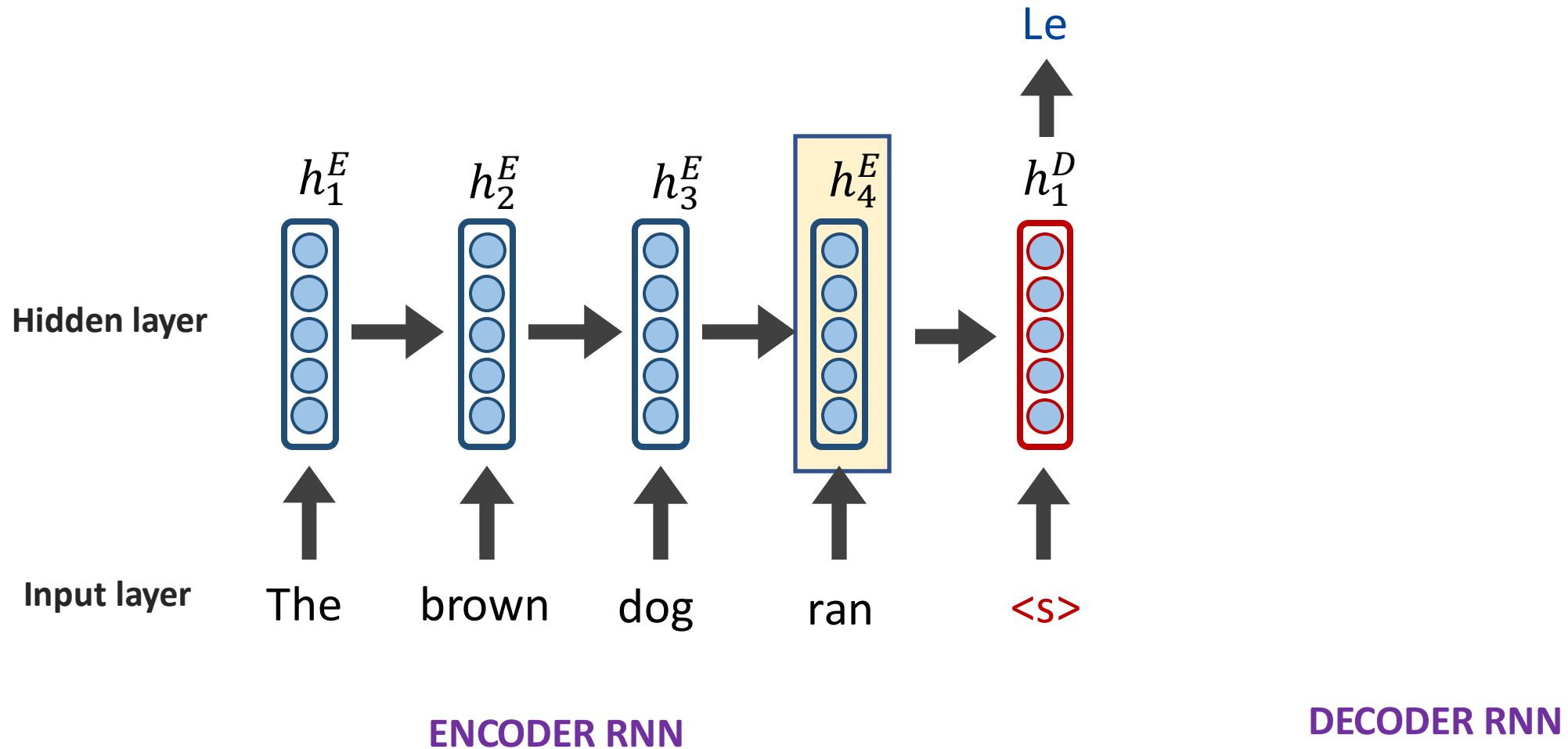
Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN



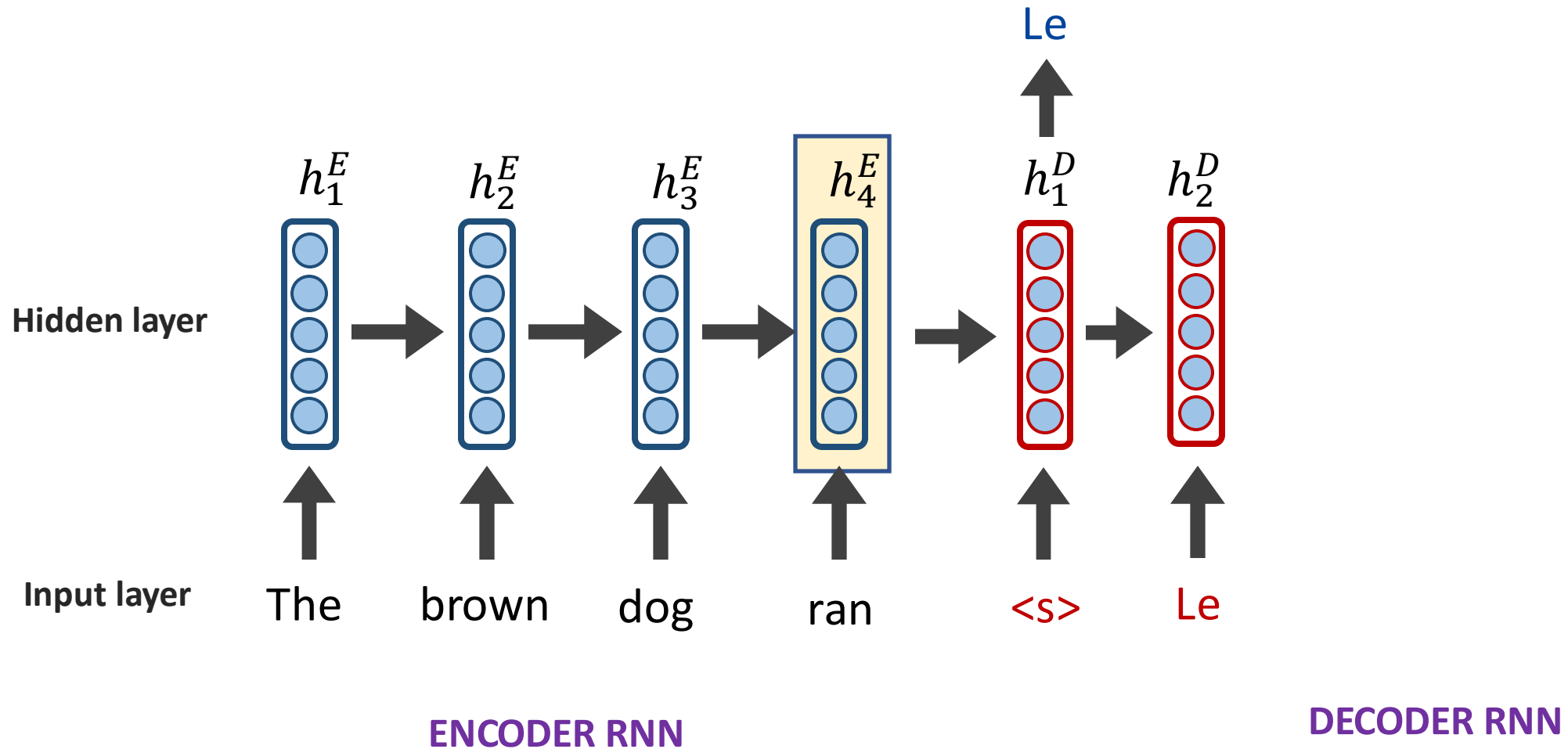
Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN



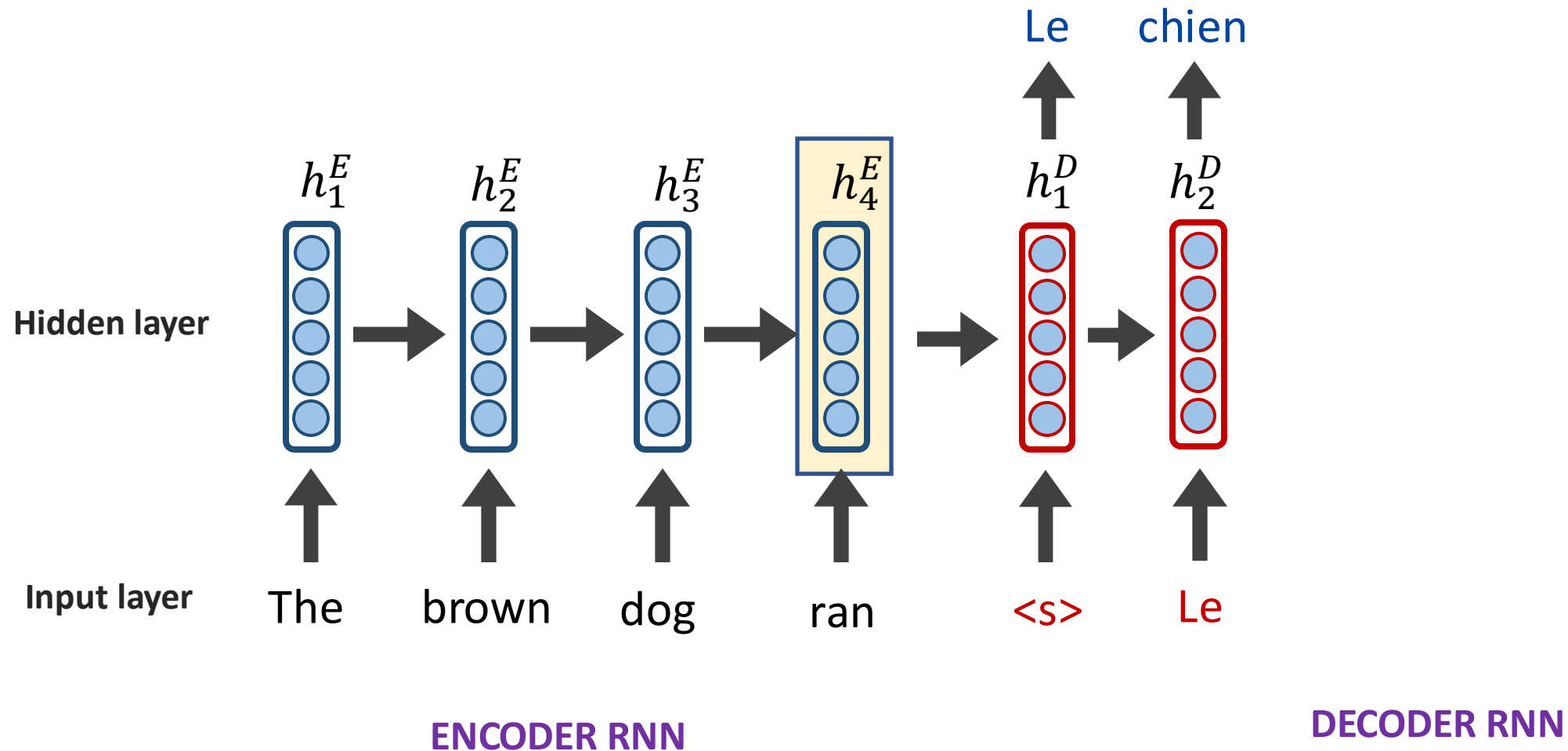
Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN



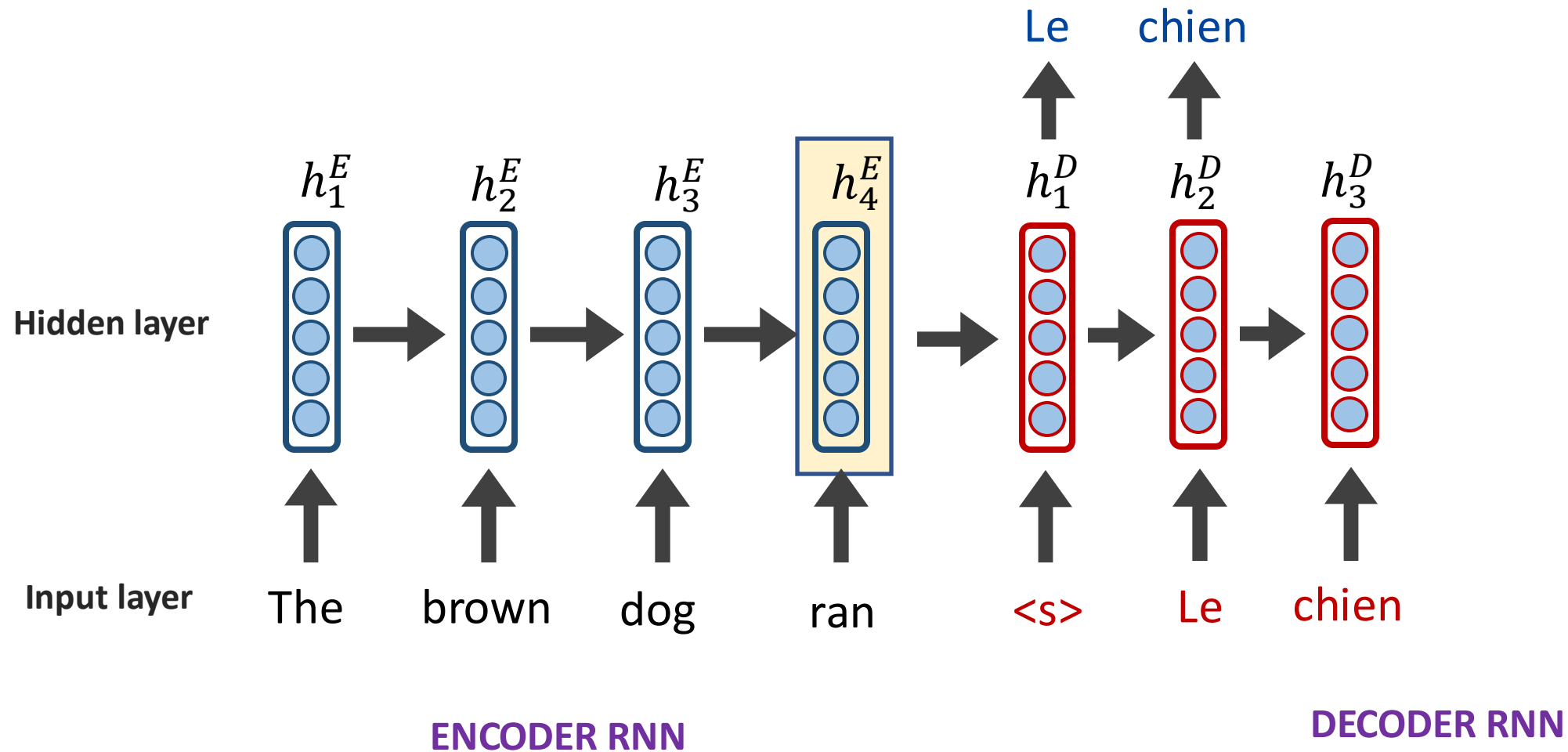
Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN



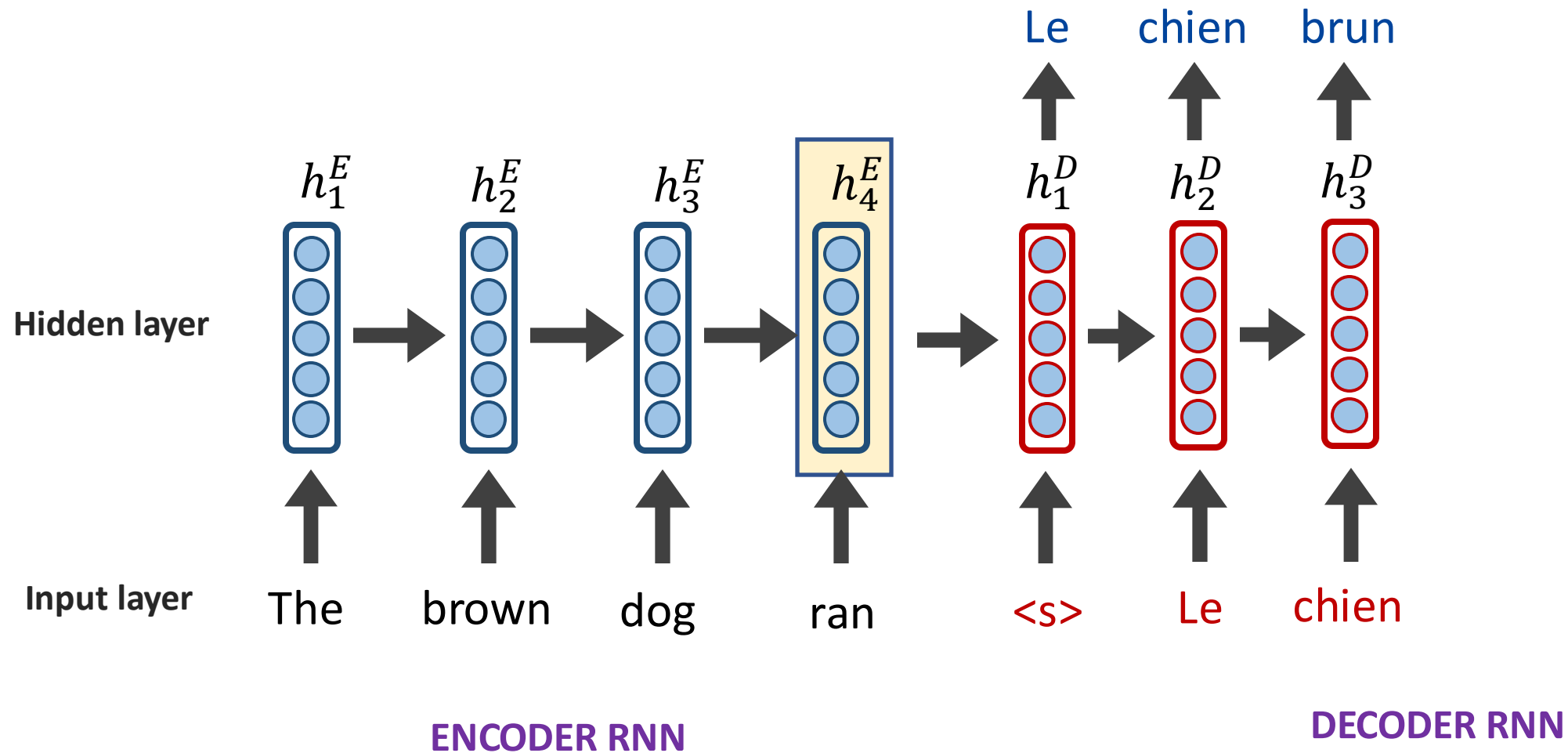
Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN



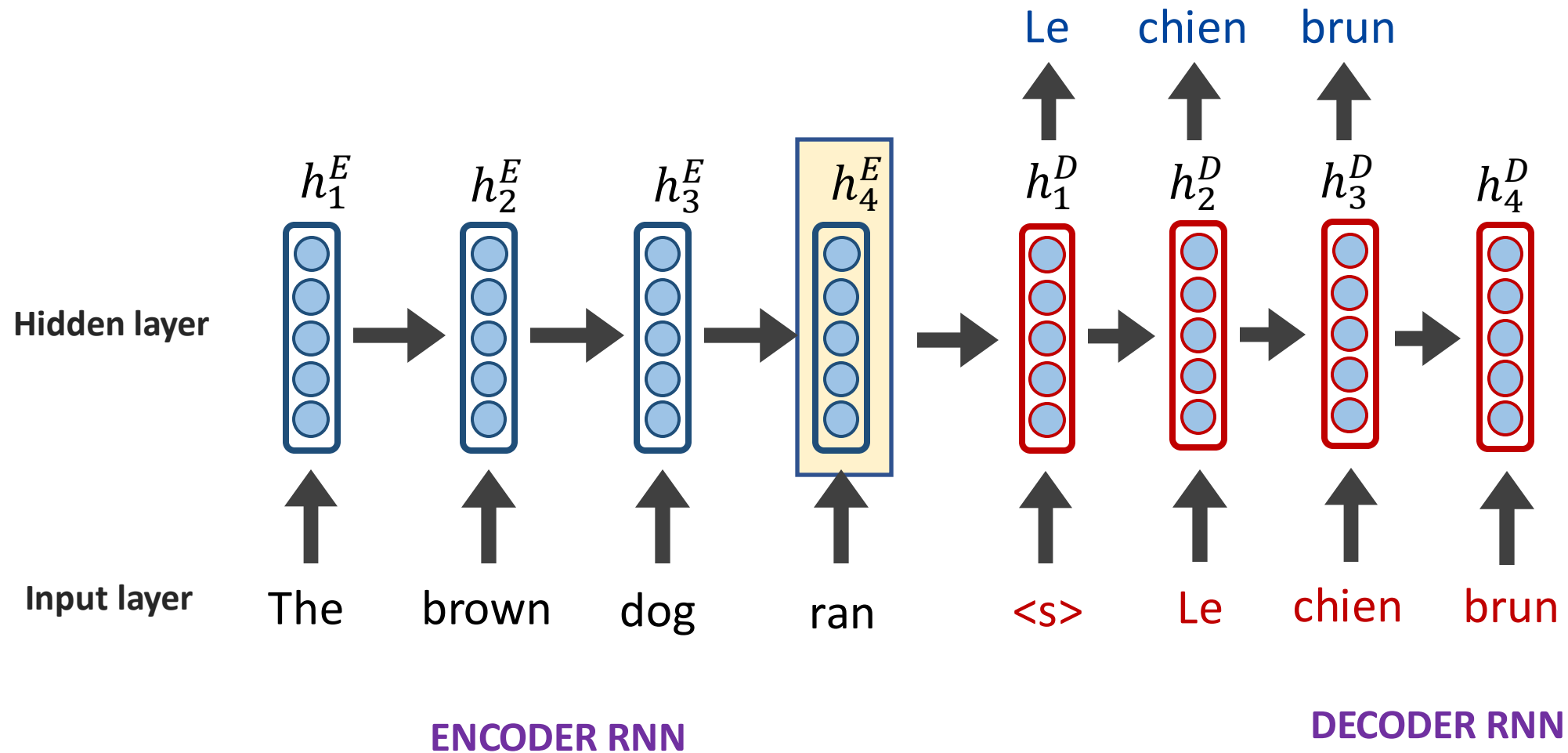
Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN



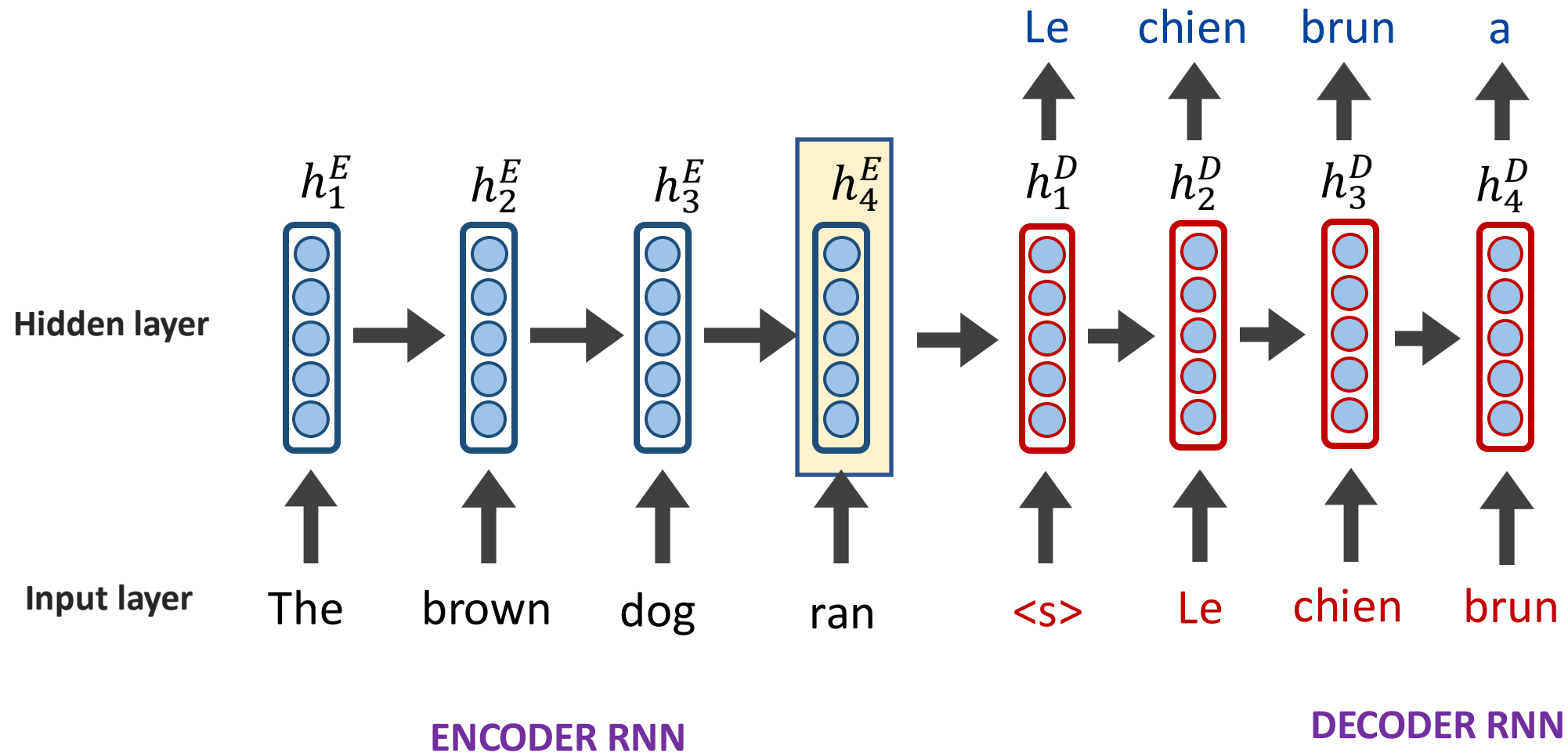
Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN



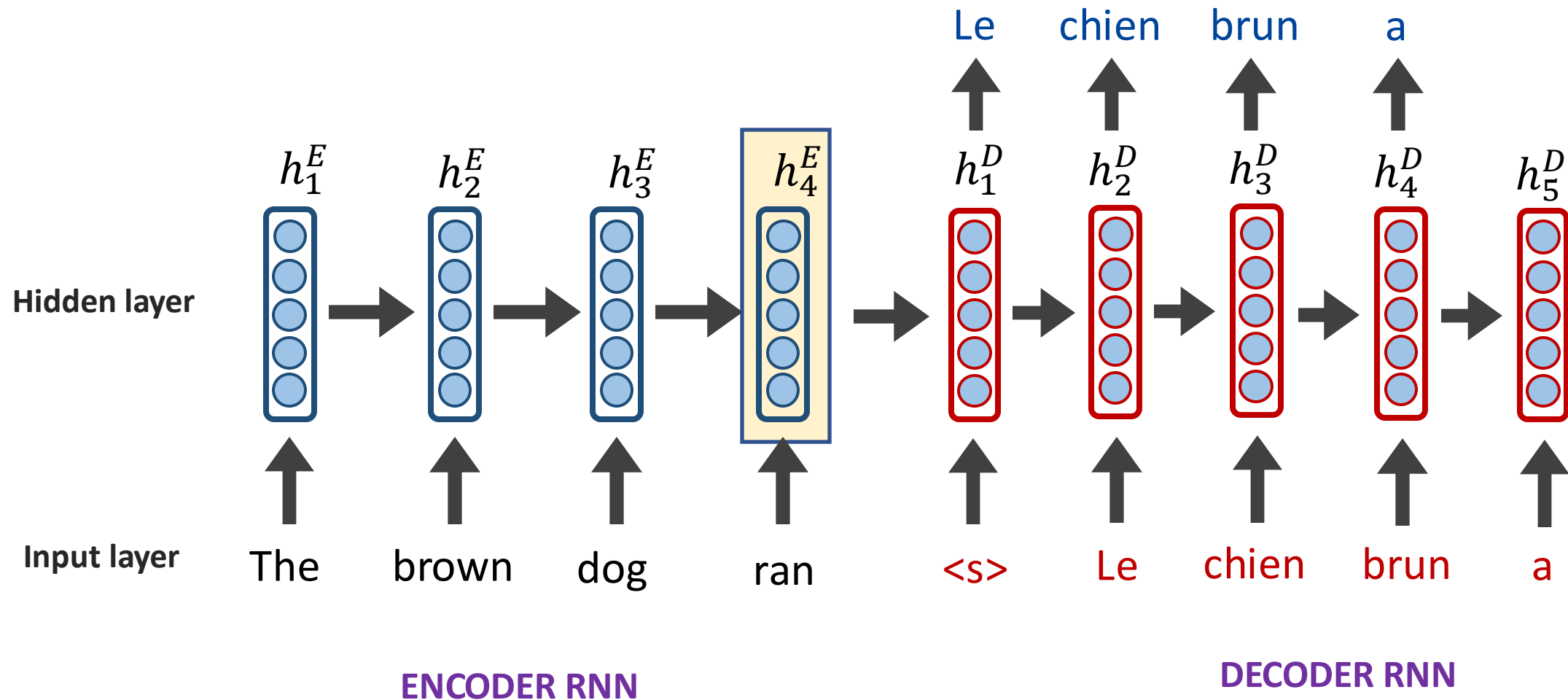
Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN



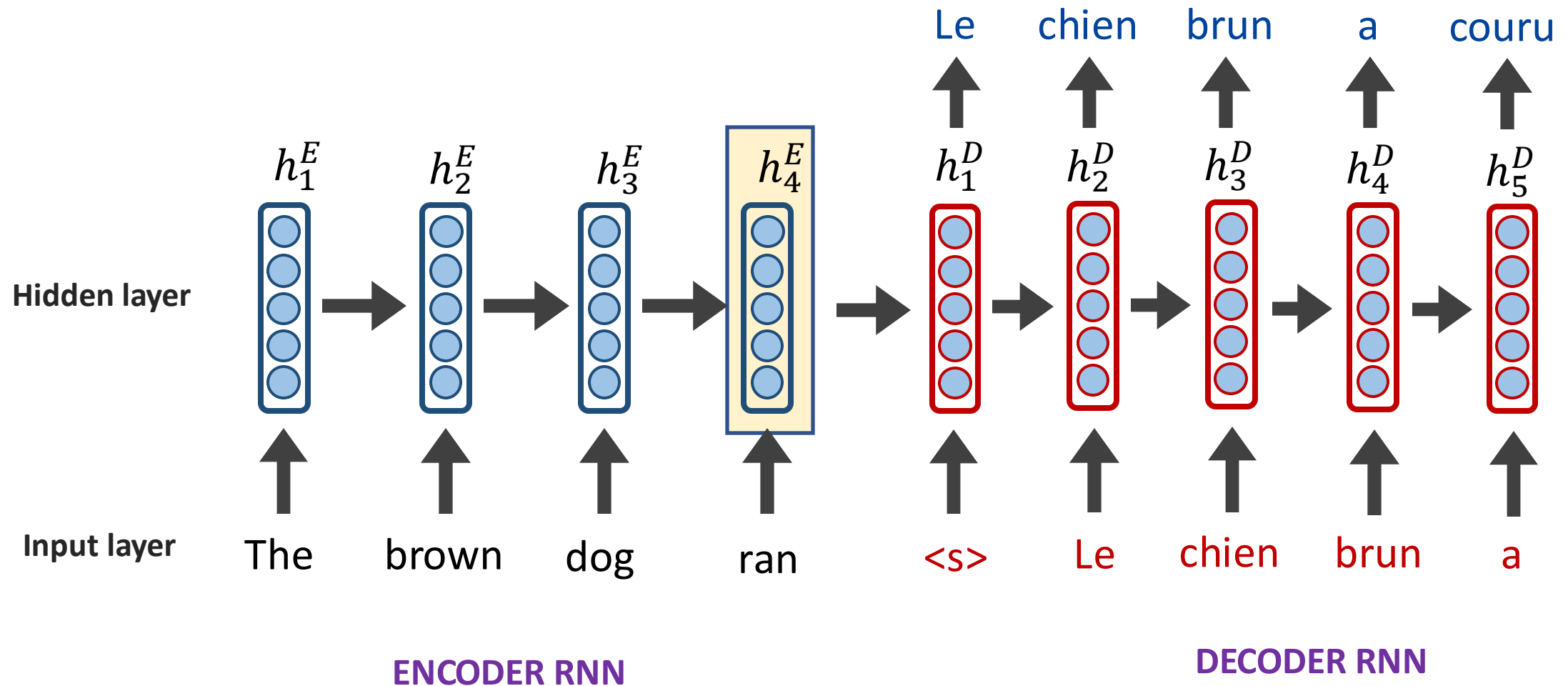
Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN



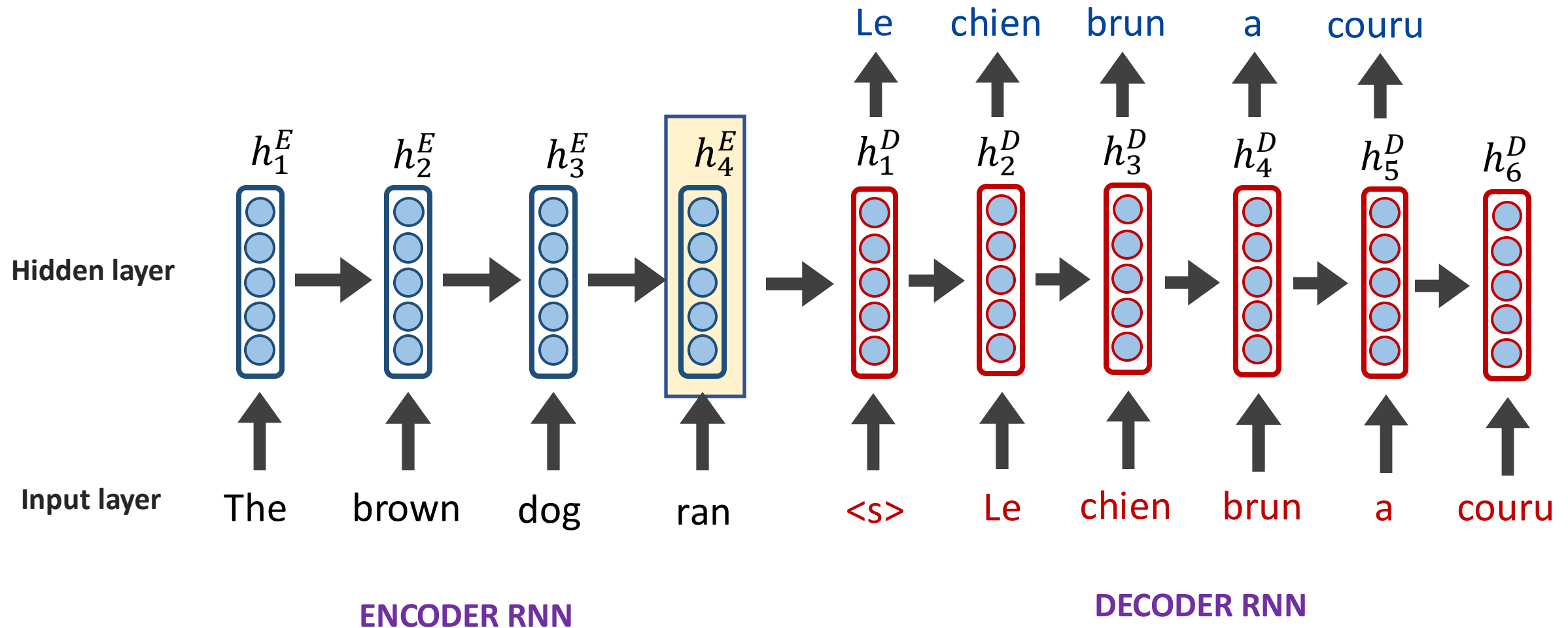
Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN



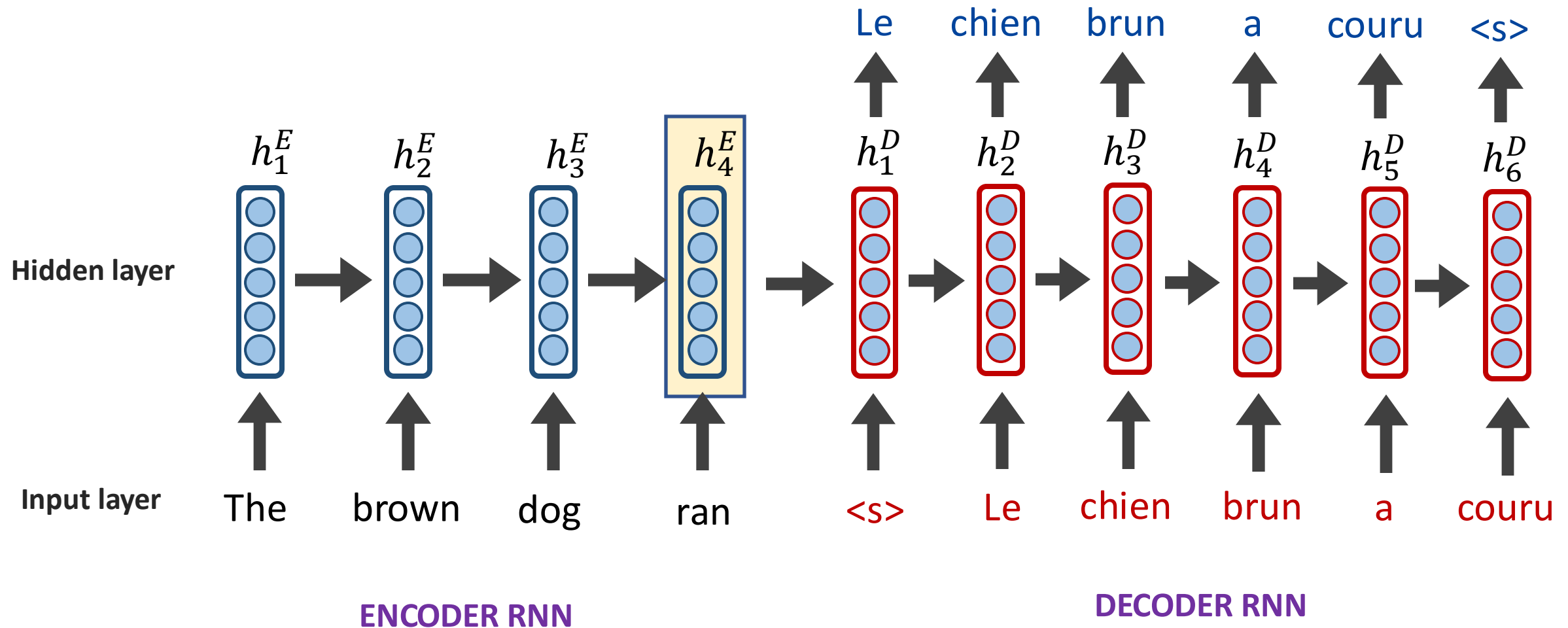
Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN

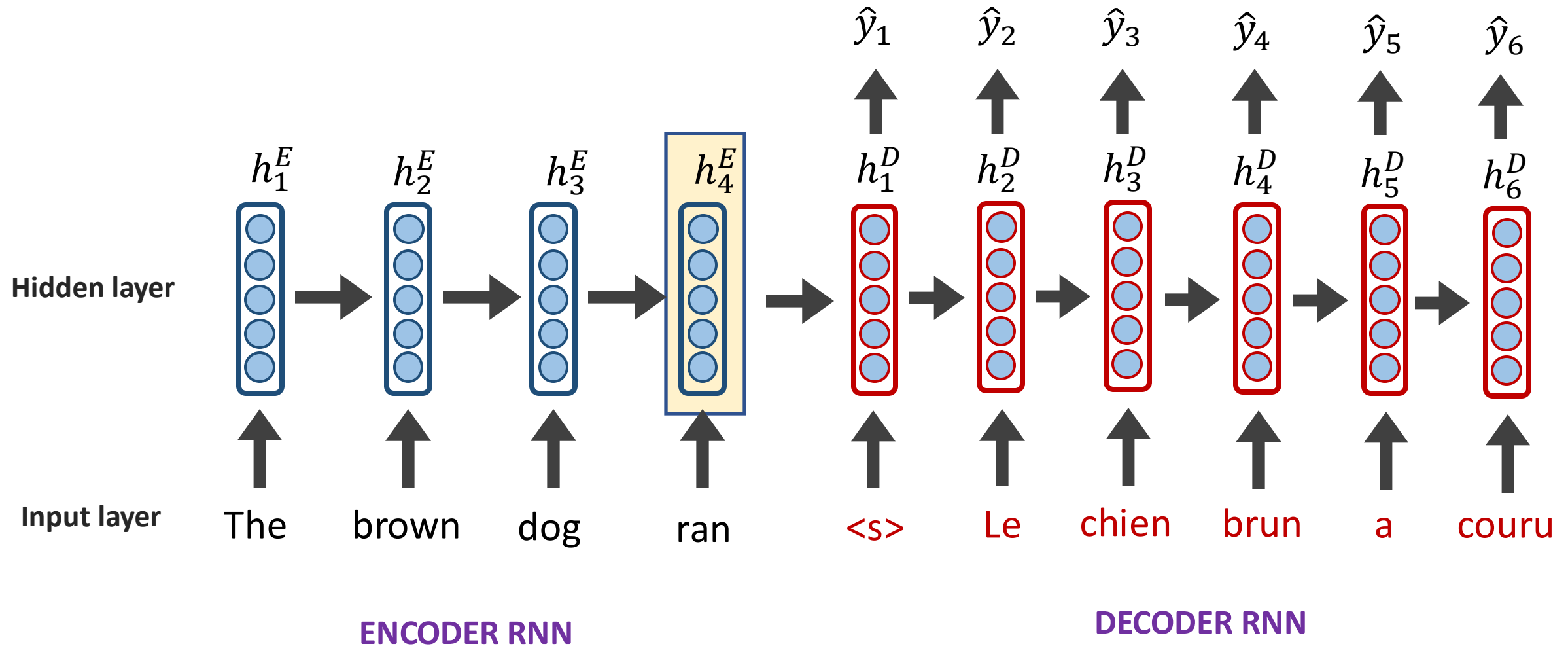


Sequence-to-Sequence (seq2seq)

The final hidden state of the encoder RNN is
the initial state of the decoder RNN

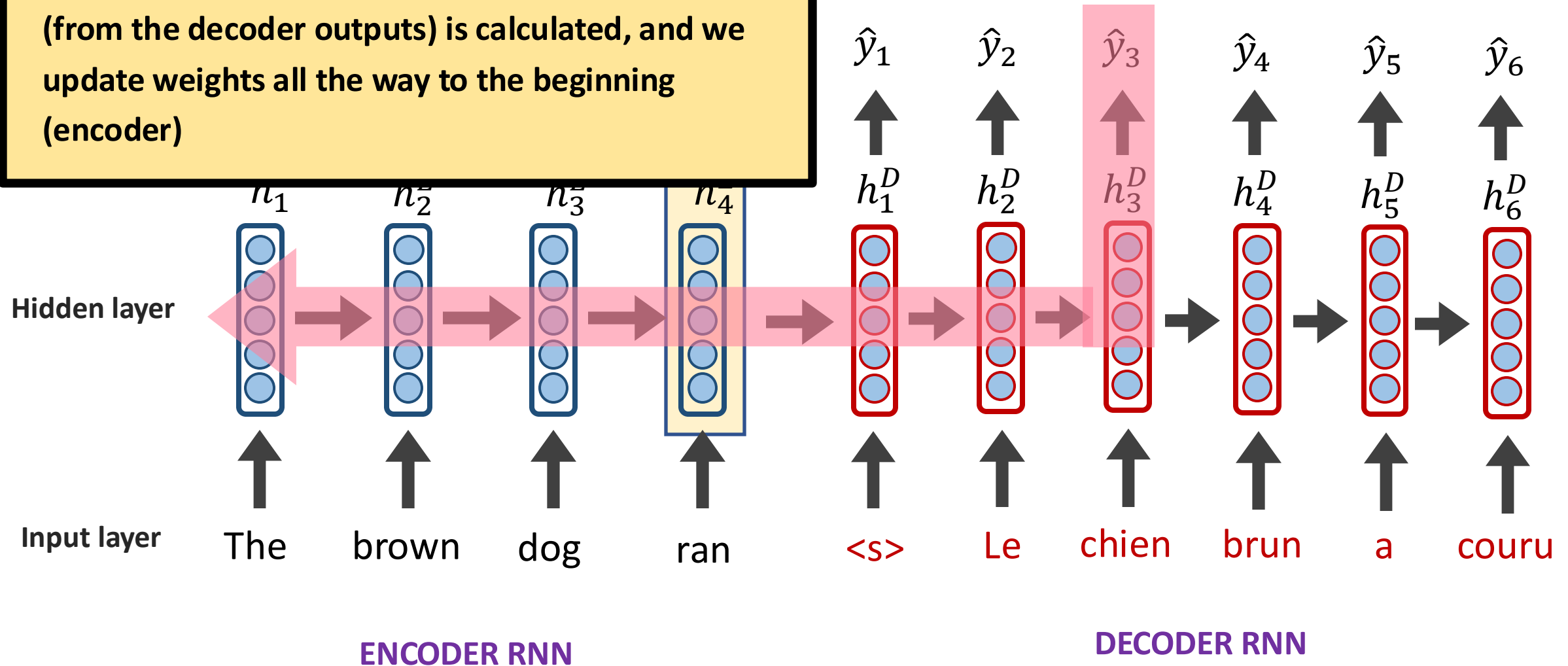


Sequence-to-Sequence (seq2seq)

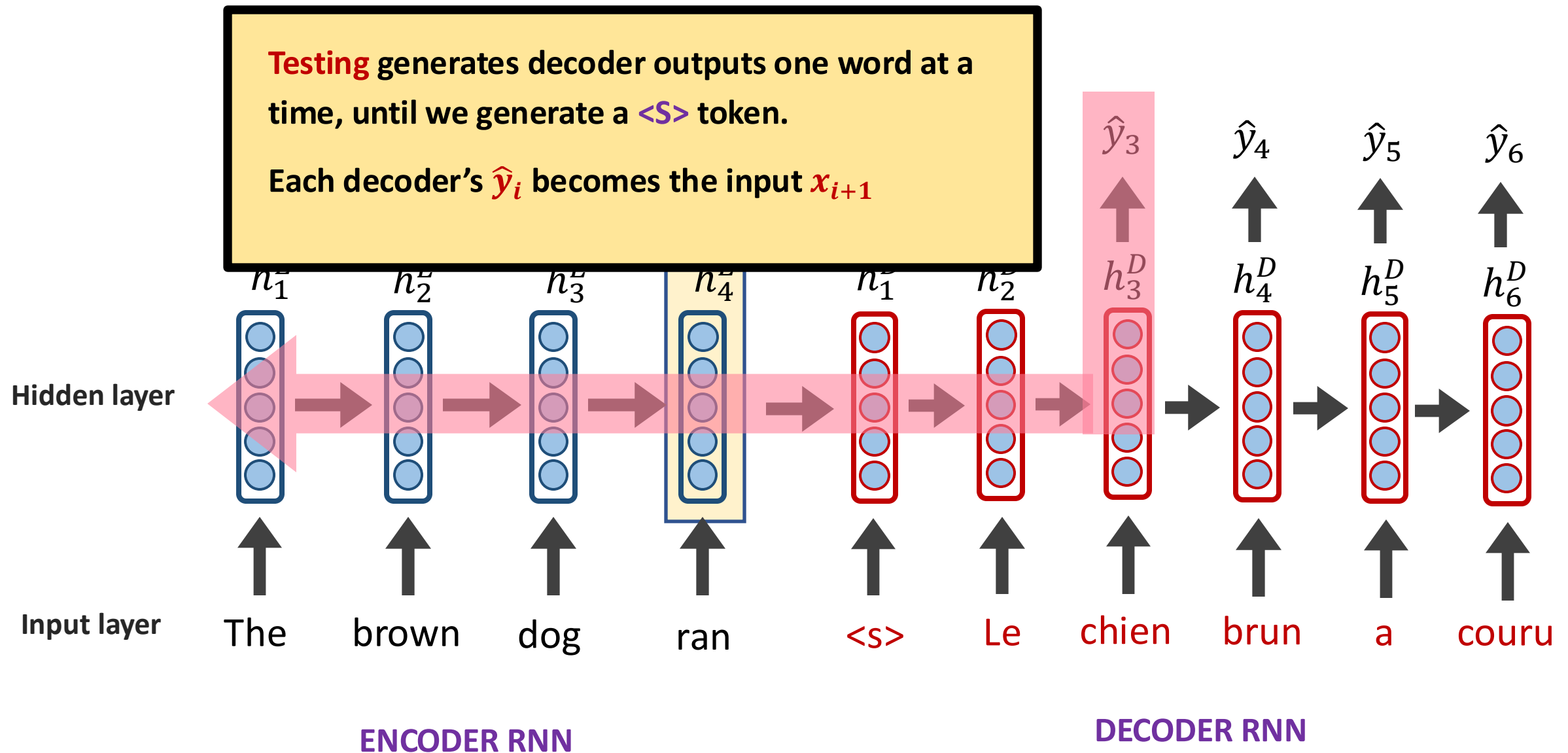


Sequence-to-Sequence (seq2seq)

Training occurs like RNNs typically do; the loss (from the decoder outputs) is calculated, and we update weights all the way to the beginning (encoder)



Sequence-to-Sequence (seq2seq)



Greedy Decoding

What's an issue w/ greedy decoding?

- "store" 0.36 (greedy pick)
 - "emergency room" 0.35
 - "emergency room" 0.35
- ## Greedy Decoding
-

She went to the _____

Next word probabilities:

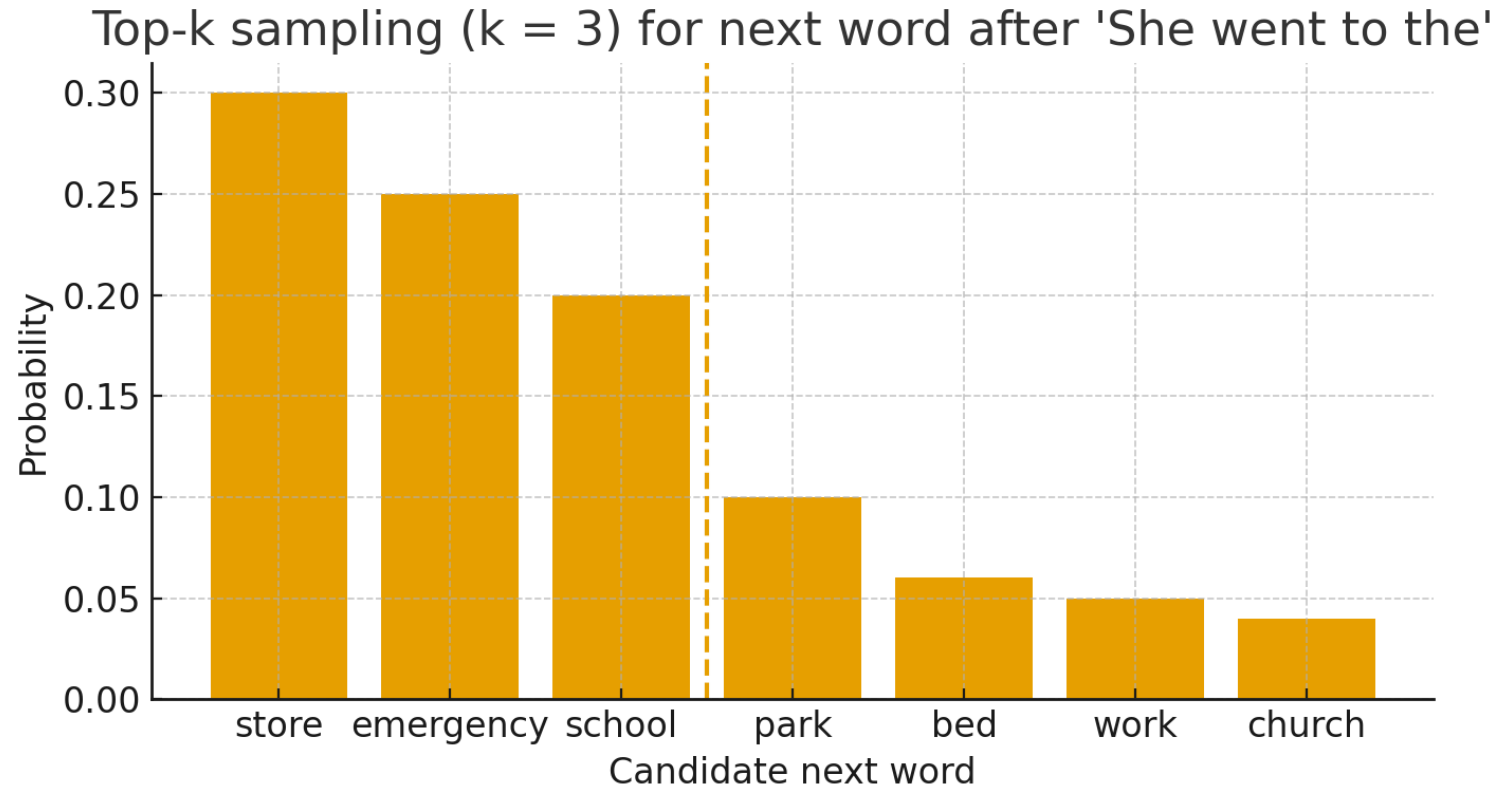
- "store" 0.36 (greedy pick)
- "emergency" 0.25

- store 0.36 (greedy pick)
 - "emergency room" 0.35
 - "emergency room" 0.35
- ## Greedy Decoding
-

She went to the **store** room because she had broken her leg.

Maybe greedy isn't always ideal!

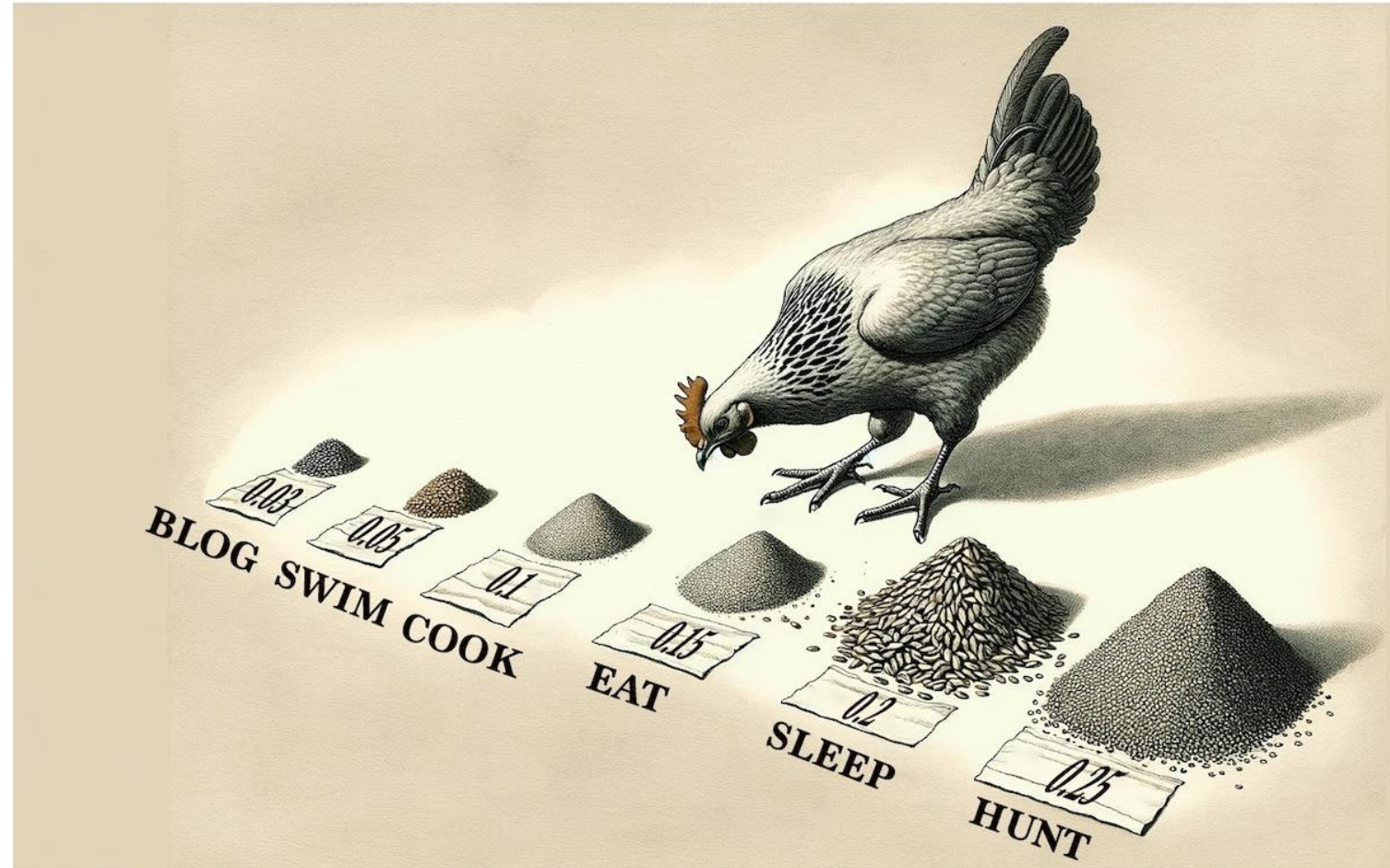
Top-k Sampling



At each step, restrict to the k most probable tokens, renormalize, and randomly sample the next word from that set.

Top-k Sampling

Secret chickens inside language models?



Neural MT

Pros:

- Better performance
- Uses context more robustly
- Better phrases
- Single model that can be optimized end-to-end (no subcomponents)
- Way less manual, feature engineering

Neural MT

Cons:

- Not too interpretable
- Hard to control/ force any Language-specific aspect
- A vanilla seq2seq approach can have gradient issues

MT Evaluation

BLEU: A similarity metric that compares the generated machine translation to a human-produced translation.

Uses n-gram precision (e.g., $n=1,2,3,4,5$)

Computer Generated: the dog

Target: the dog ran fast

MT Evaluation

BLEU: A similarity metric that compares the generated machine translation to a human-produced translation.

Uses n-gram precision (e.g., $n=1,2,3,4,5$)

Computer Generated: the dog

Target: the dog ran fast

Adds a penalty for translations that are too short (akin to **recall**)

2014 - present: NMT

SUMMARY

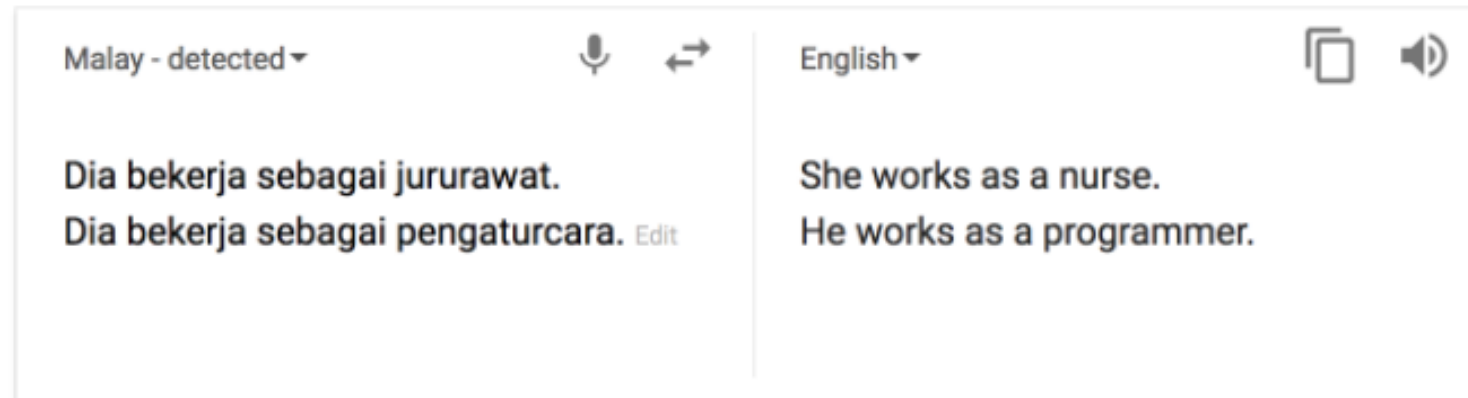
- Became SOTA in just 2 years
- OOV issues still need to be handled
- Susceptible to training data, as always (domain mismatch issues)
- Long-context is always difficult
- Low-resource languages still remains a challenge
- Biases from training data

MT Biases

Level 1: No context provided.

Both Google Translate and Microsoft Translator return

- **she** for **nurse**
- **he** for **programmer**



Both Google and Microsoft return same result.

MT Biases

Detect language Malay **English** French ▼ ↔ **Malay** French English ▼

he is a nurse

13 / 5,000

dia seorang jururawat

Send feedback

Click the swap languages button ↔

Detect language **Malay** English French ▼ ↔ Malay French **English** ▼

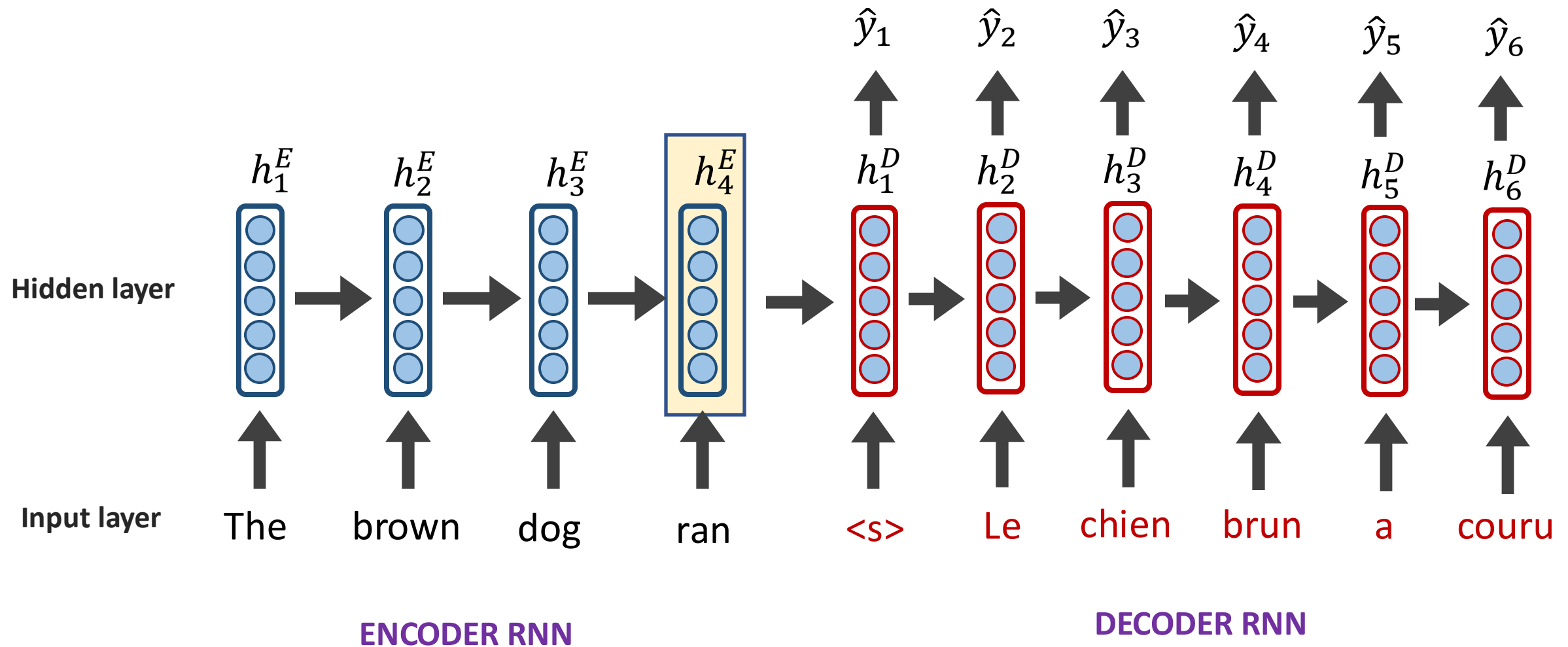
dia seorang jururawat

21 / 5,000

she is a nurse

Send feedback

Sequence-to-Sequence (seq2seq)

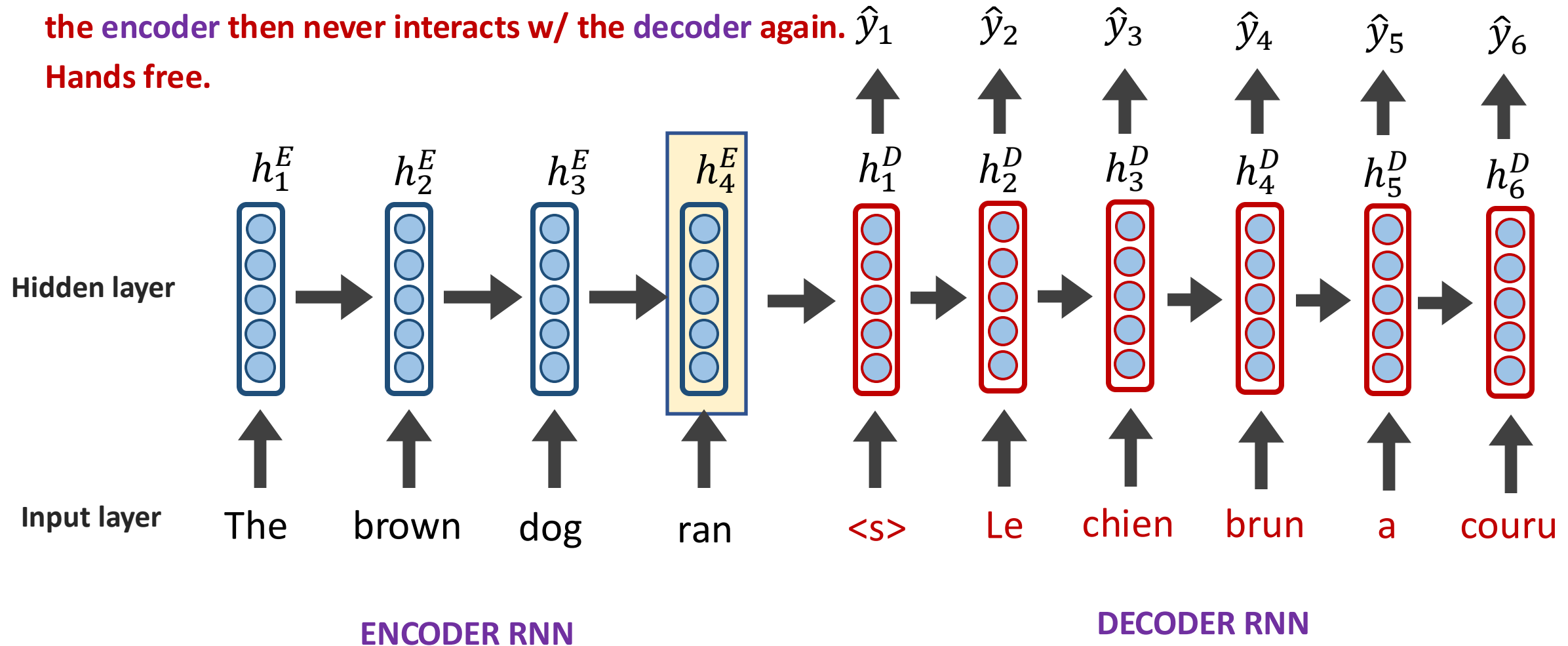


Sequence-to-Sequence (seq2seq)

See any issues with this traditional **seq2seq** paradigm?

Sequence-to-Sequence (seq2seq)

It's crazy that the entire "meaning" of the 1st sequence is expected to be packed into this **one embedding**, and that the encoder then never interacts w/ the decoder again. Hands free.



Sequence-to-Sequence (seq2seq)

Instead, what if the decoder, at each step, pays **attention** to a *distribution* of all of the encoder's hidden states?

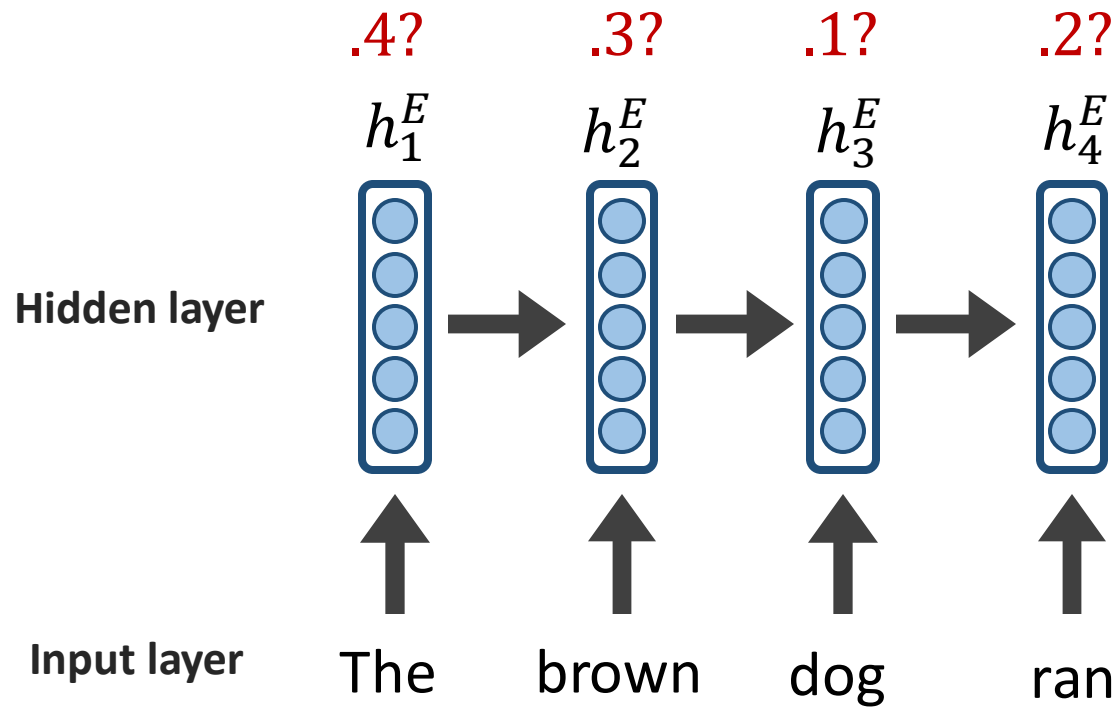
Sequence-to-Sequence (seq2seq)

Instead, what if the decoder, at each step, pays **attention** to a *distribution* of all of the encoder's hidden states?

Intuition: when we (humans) translate a sentence, we don't just consume the original sentence, reflect on the meaning of the last word, then regurgitate in a new language; we **continuously think back at the original sentence** while focusing on **different parts**.

seq2seq + Attention

Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

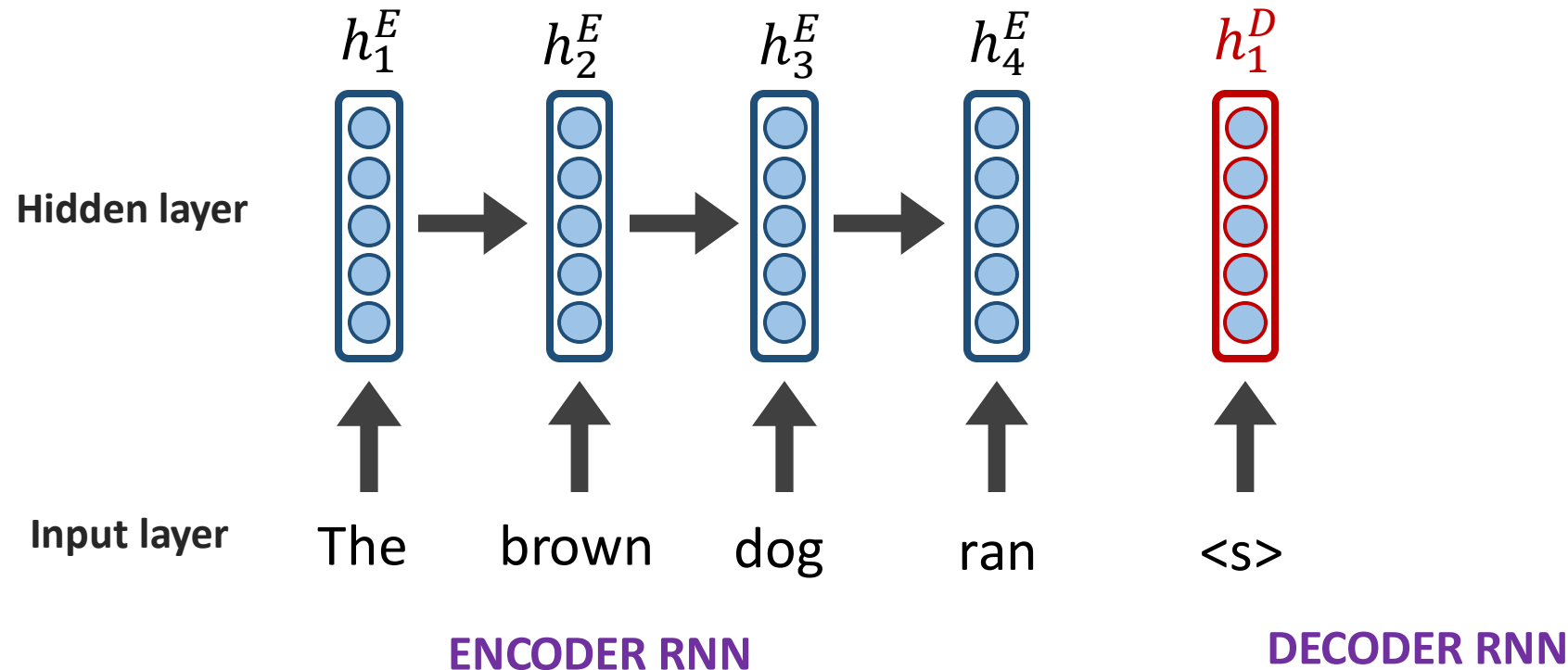


ENCODER RNN

seq2seq + Attention

Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

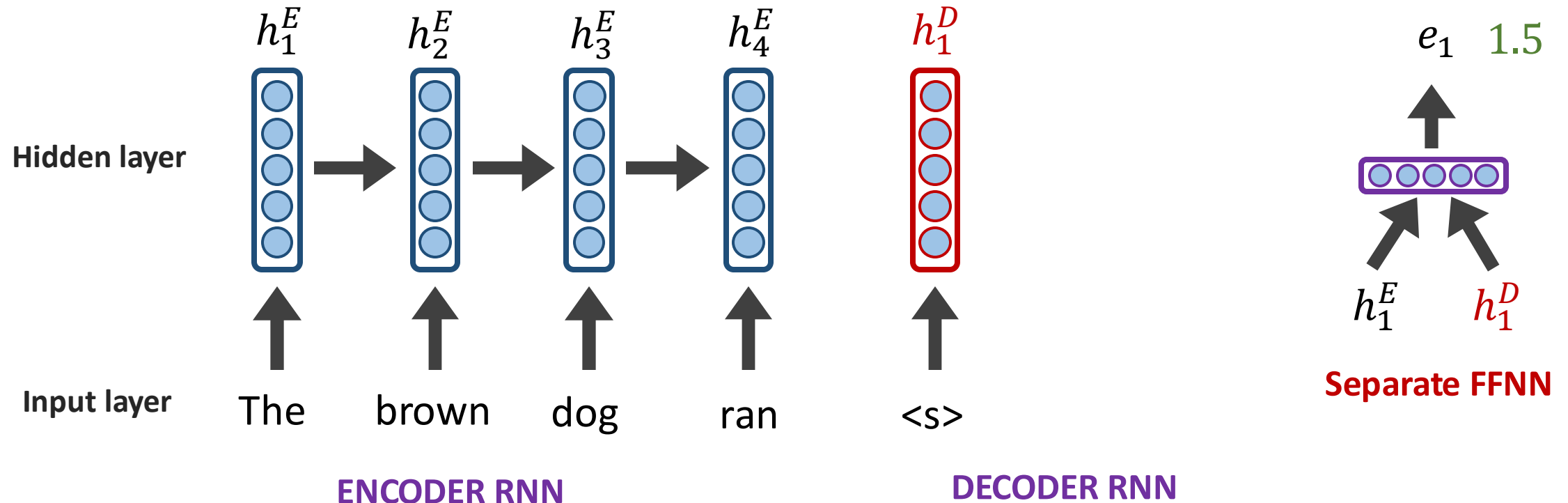
A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



seq2seq + Attention

Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

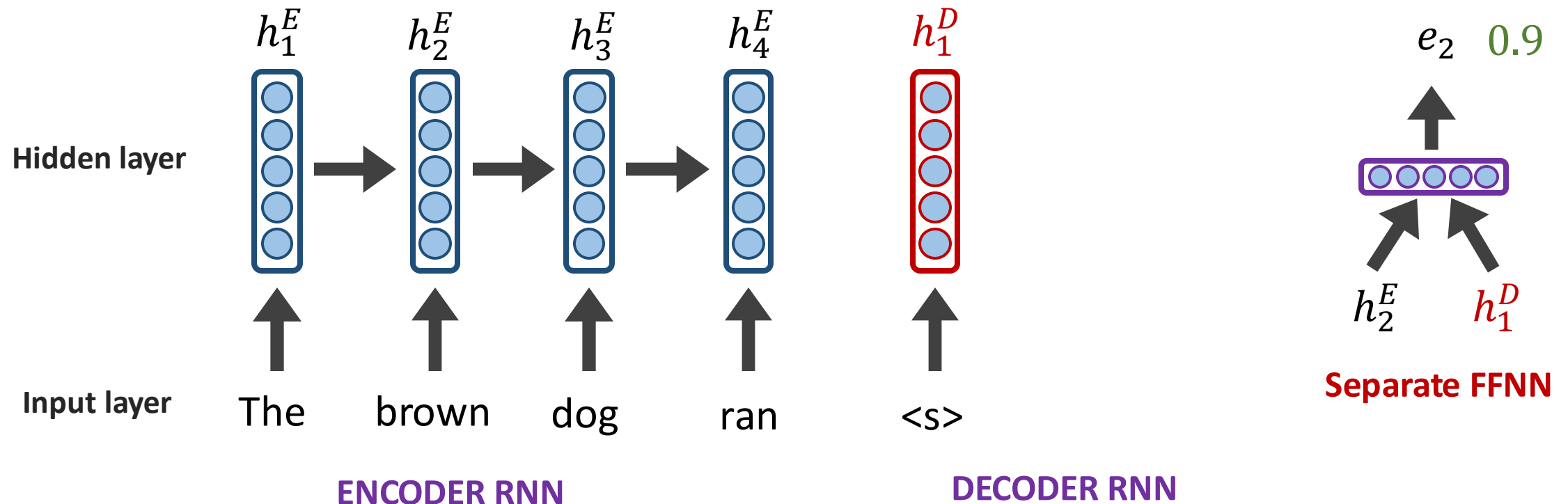
A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



seq2seq + Attention

Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

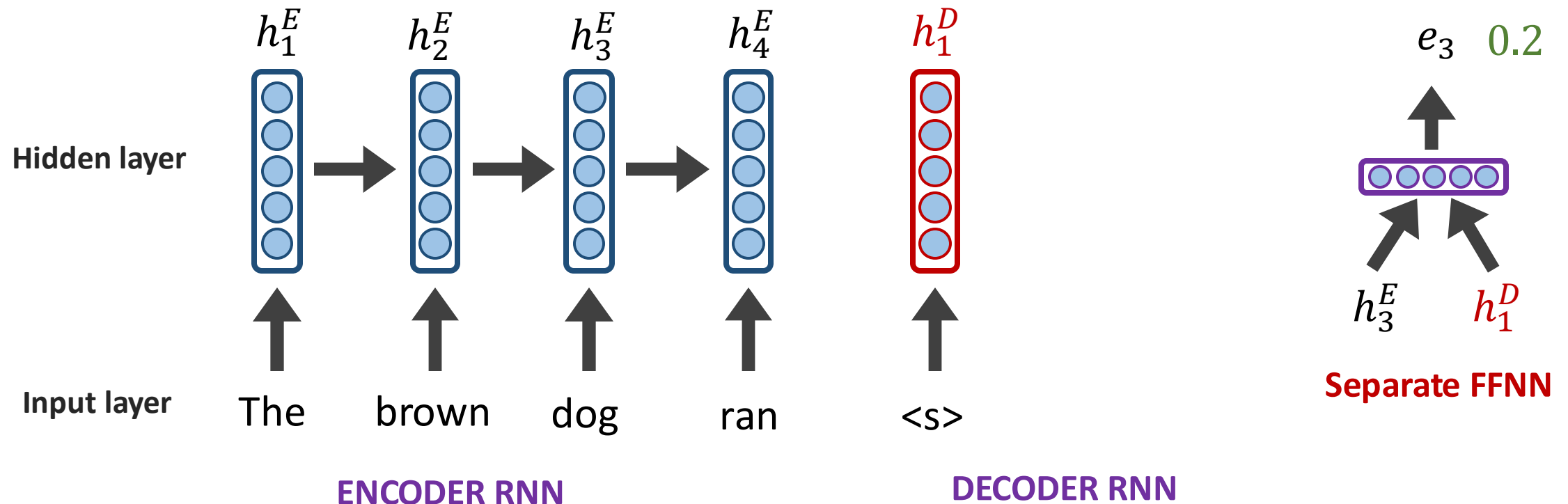
A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



seq2seq + Attention

Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

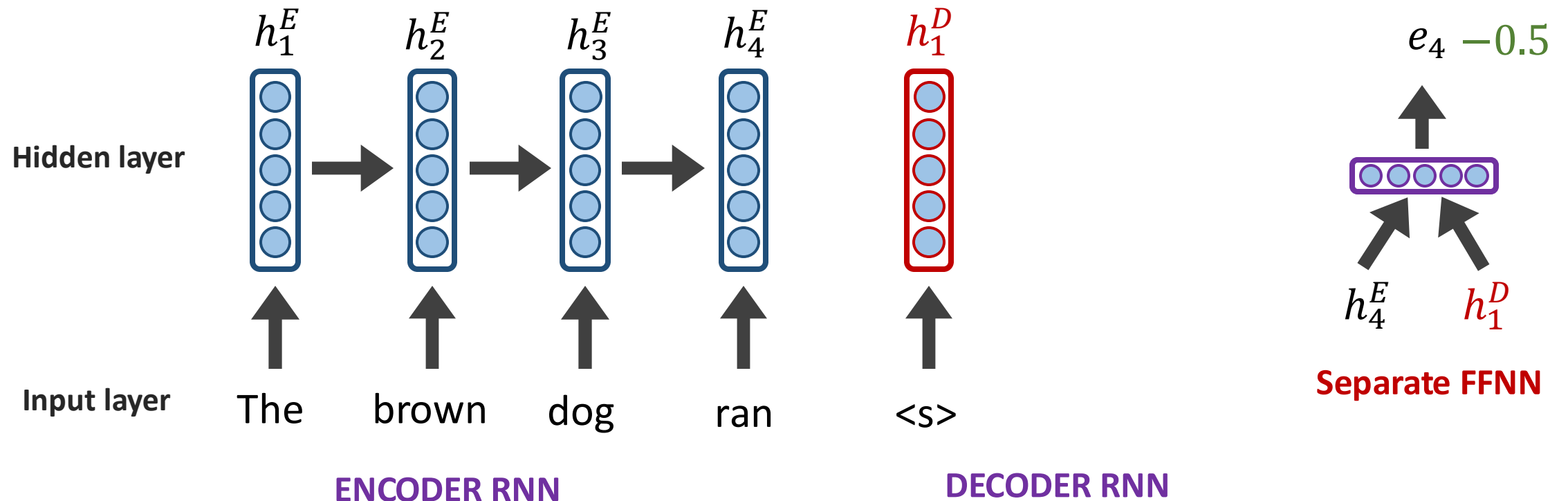
A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



seq2seq + Attention

Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

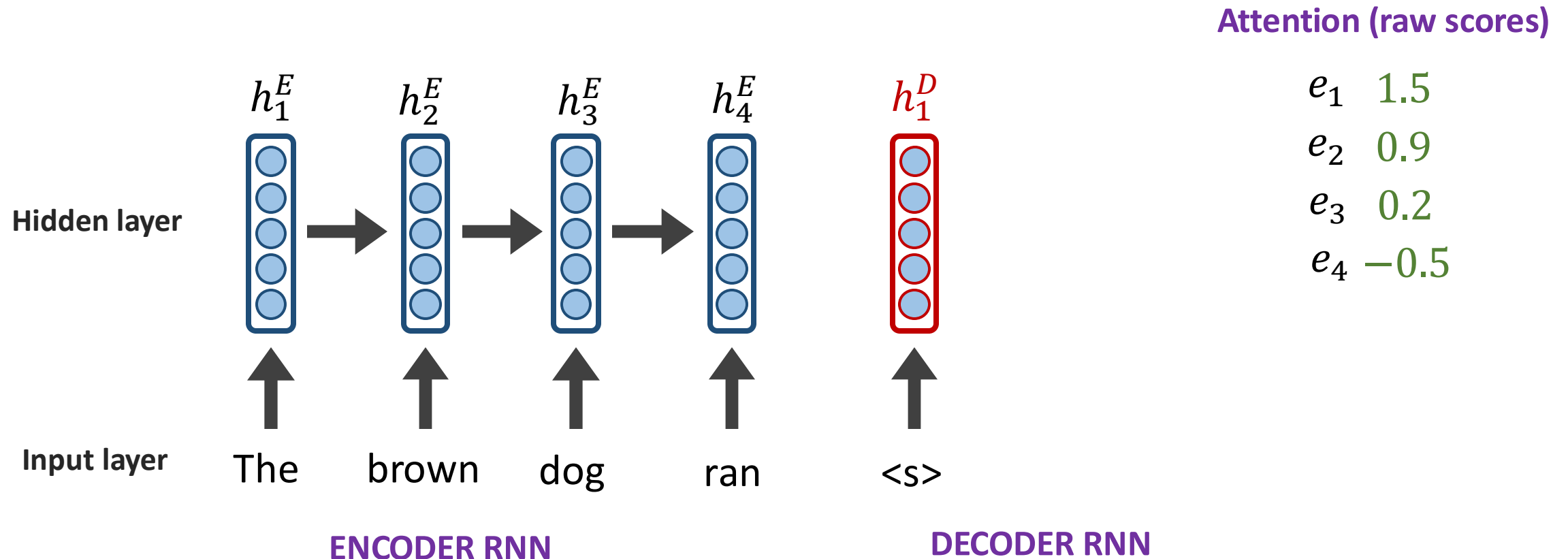
A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



seq2seq + Attention

Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

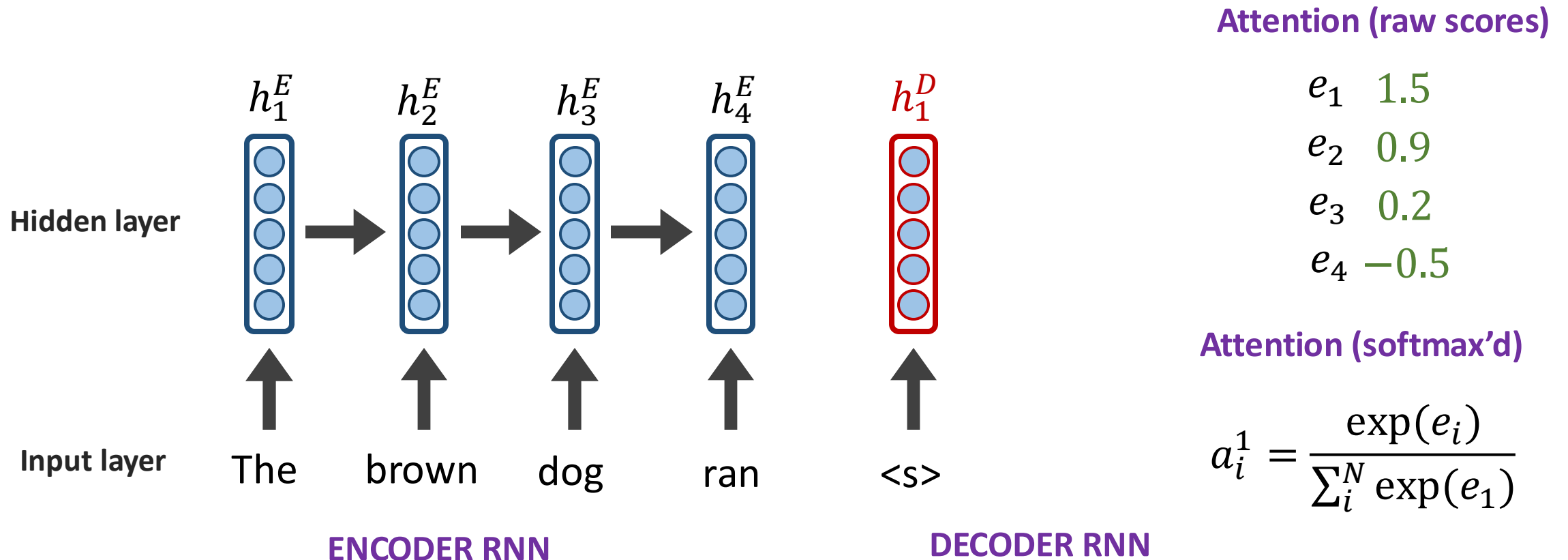
A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



seq2seq + Attention

Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

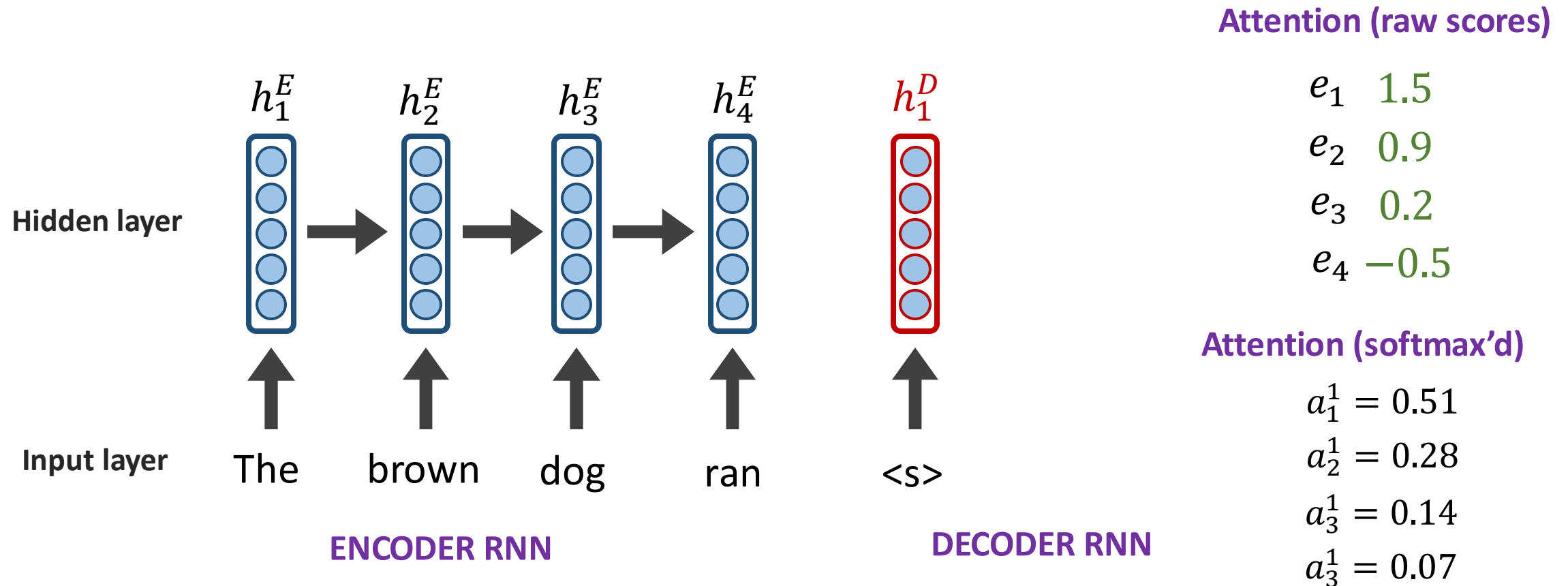
A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



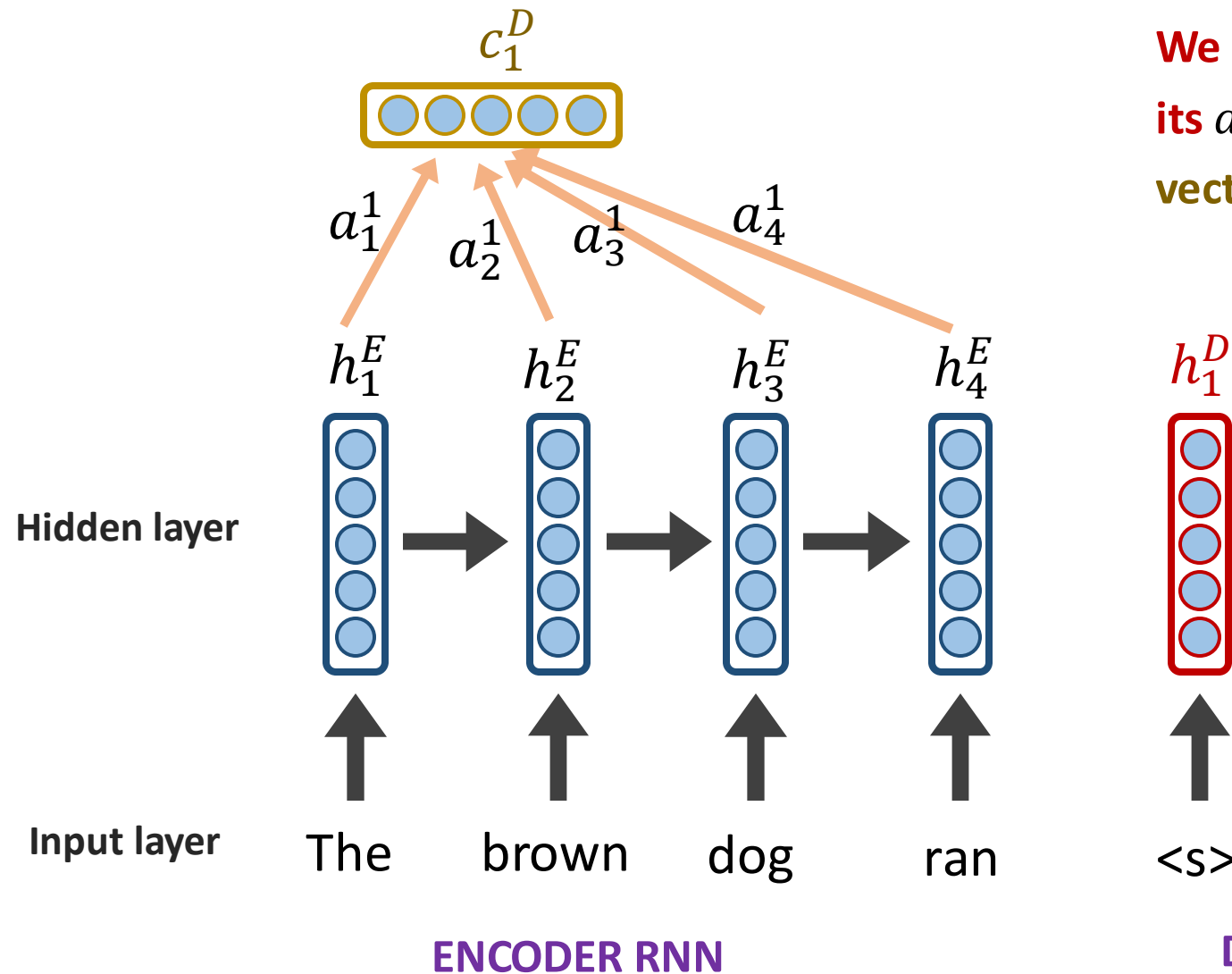
seq2seq + Attention

Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



seq2seq + Attention



We multiply each encoder's hidden layer by its a_i^1 attention weights to create a context vector c_1^D

Attention (softmax'd)

$$a_1^1 = 0.51$$

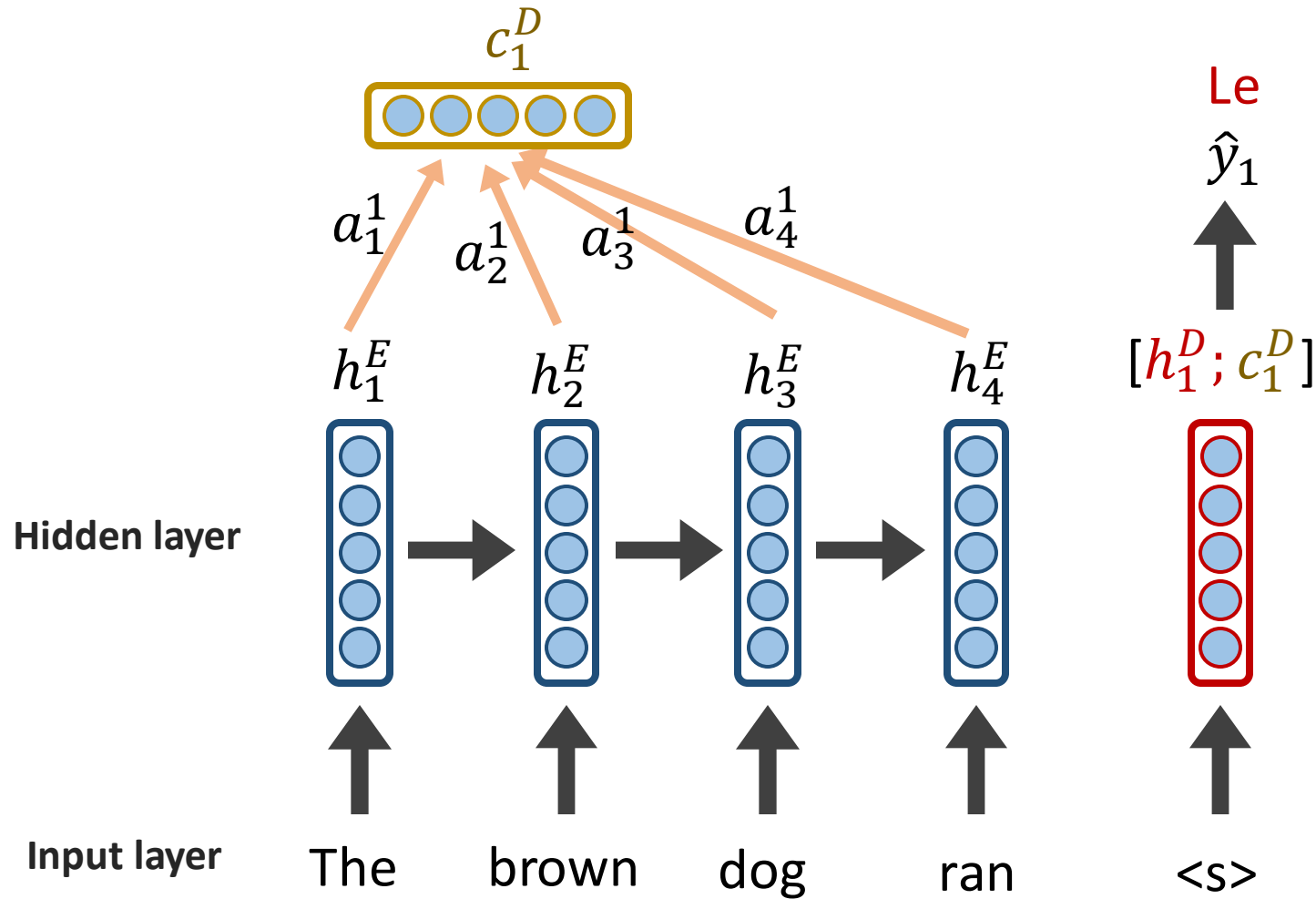
$$a_2^1 = 0.28$$

$$a_3^1 = 0.14$$

$$a_4^1 = 0.07$$

seq2seq + Attention

REMEMBER: each attention weight a_i^j is based on the **decoder's** current hidden state, too.

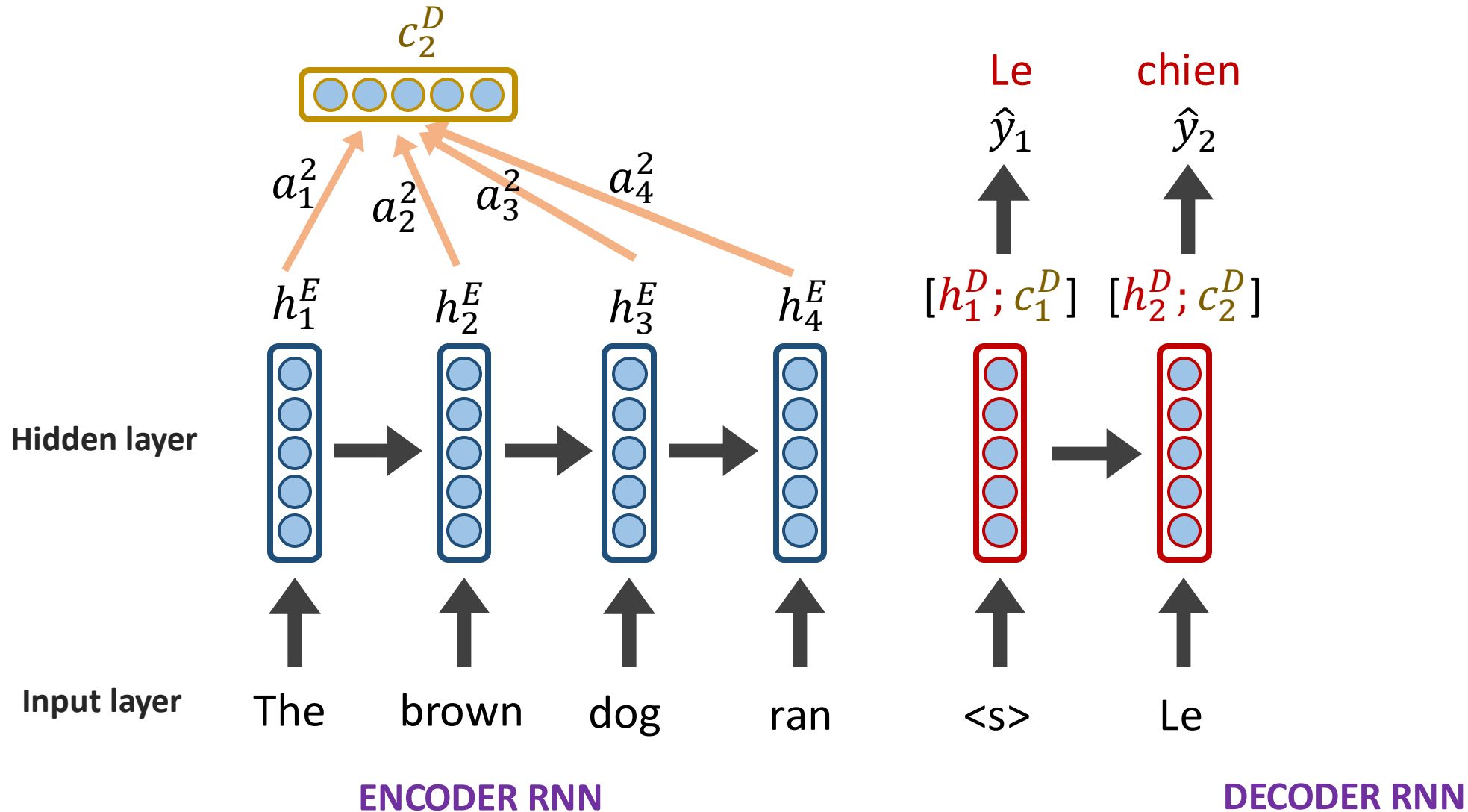


ENCODER RNN

DECODER RNN

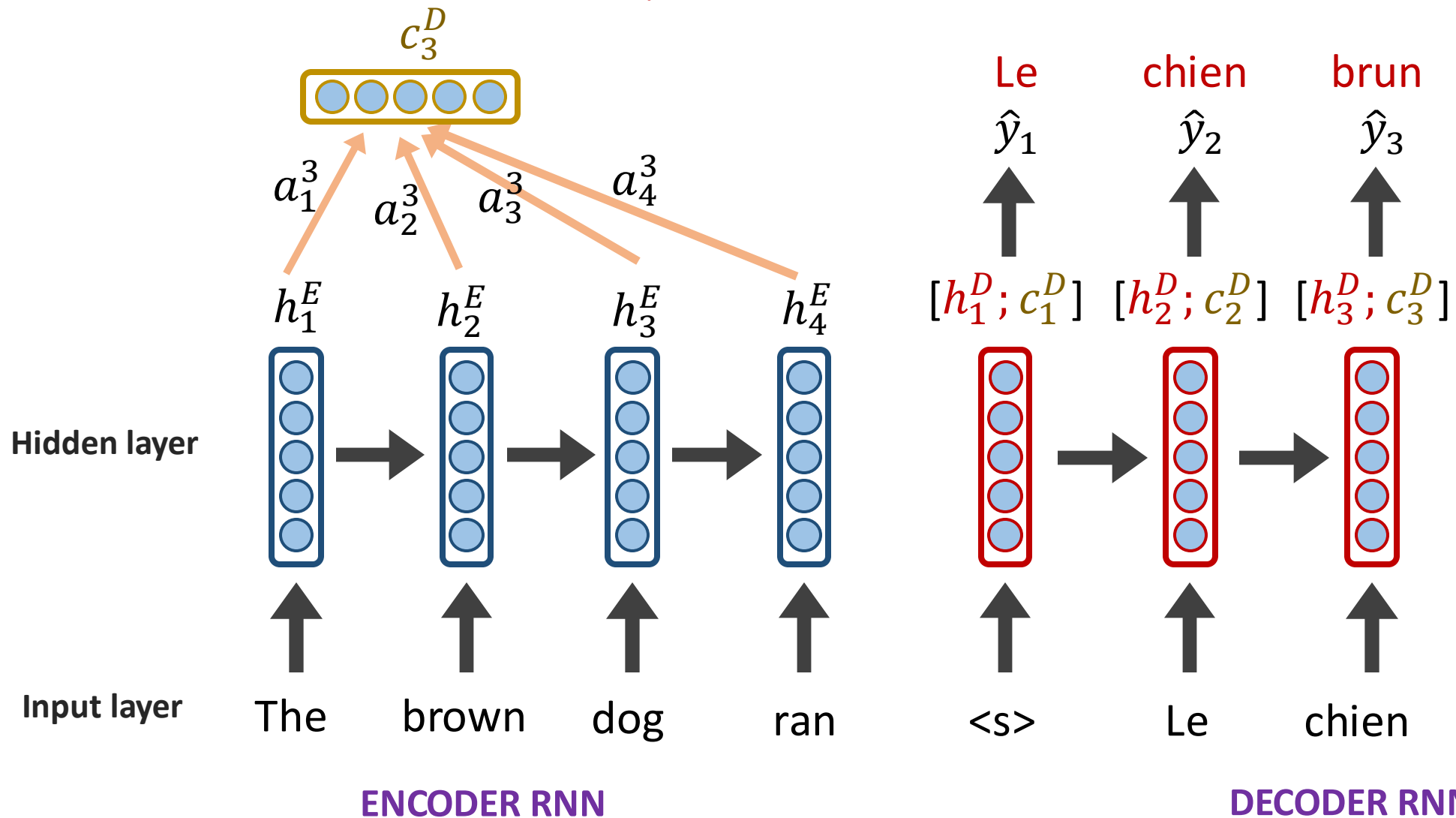
seq2seq + Attention

REMEMBER: each attention weight a_i^j is based on the **decoder's** current hidden state, too.



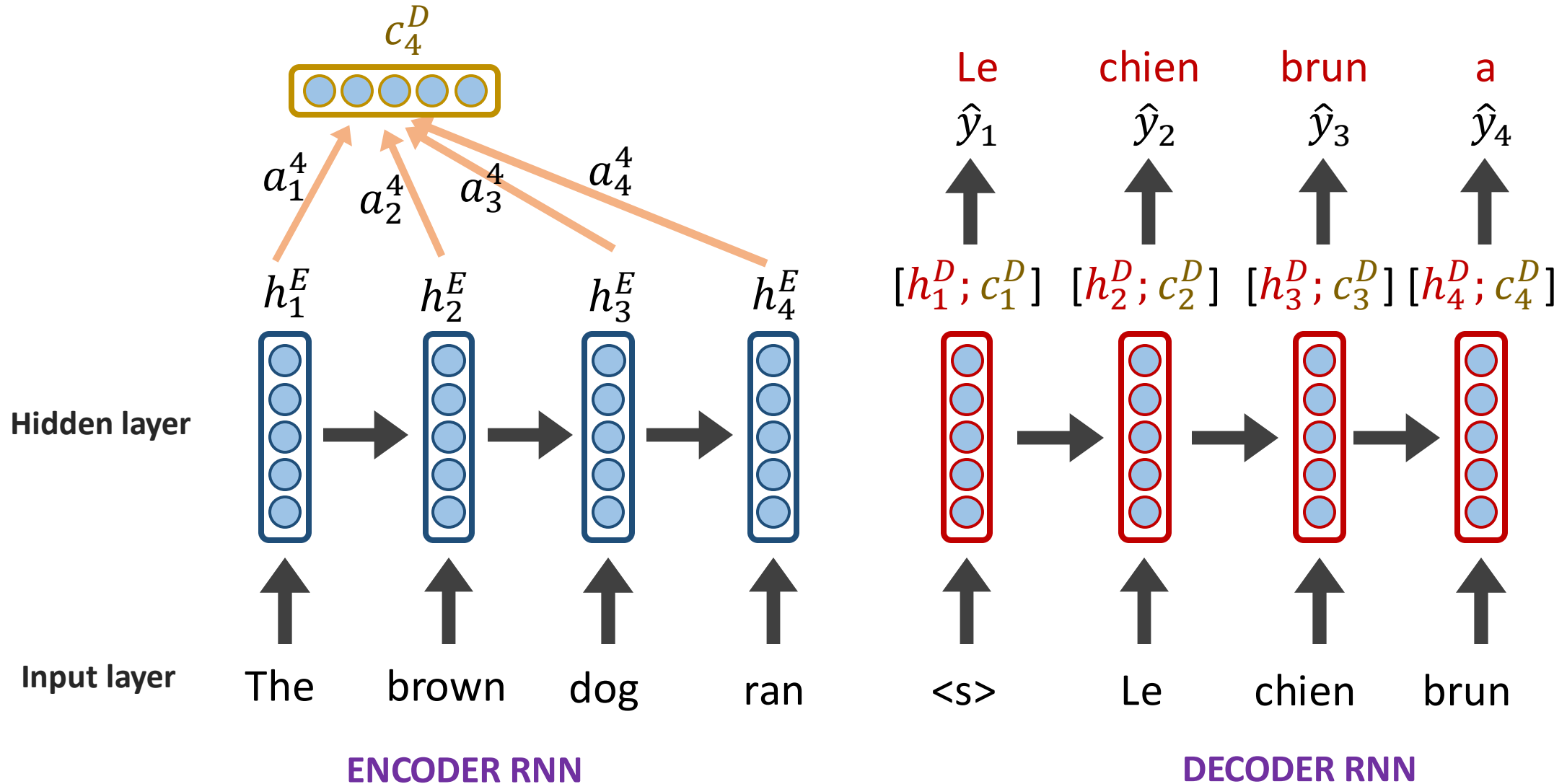
seq2seq + Attention

REMEMBER: each attention weight a_i^j is based on the **decoder's** current hidden state, too.



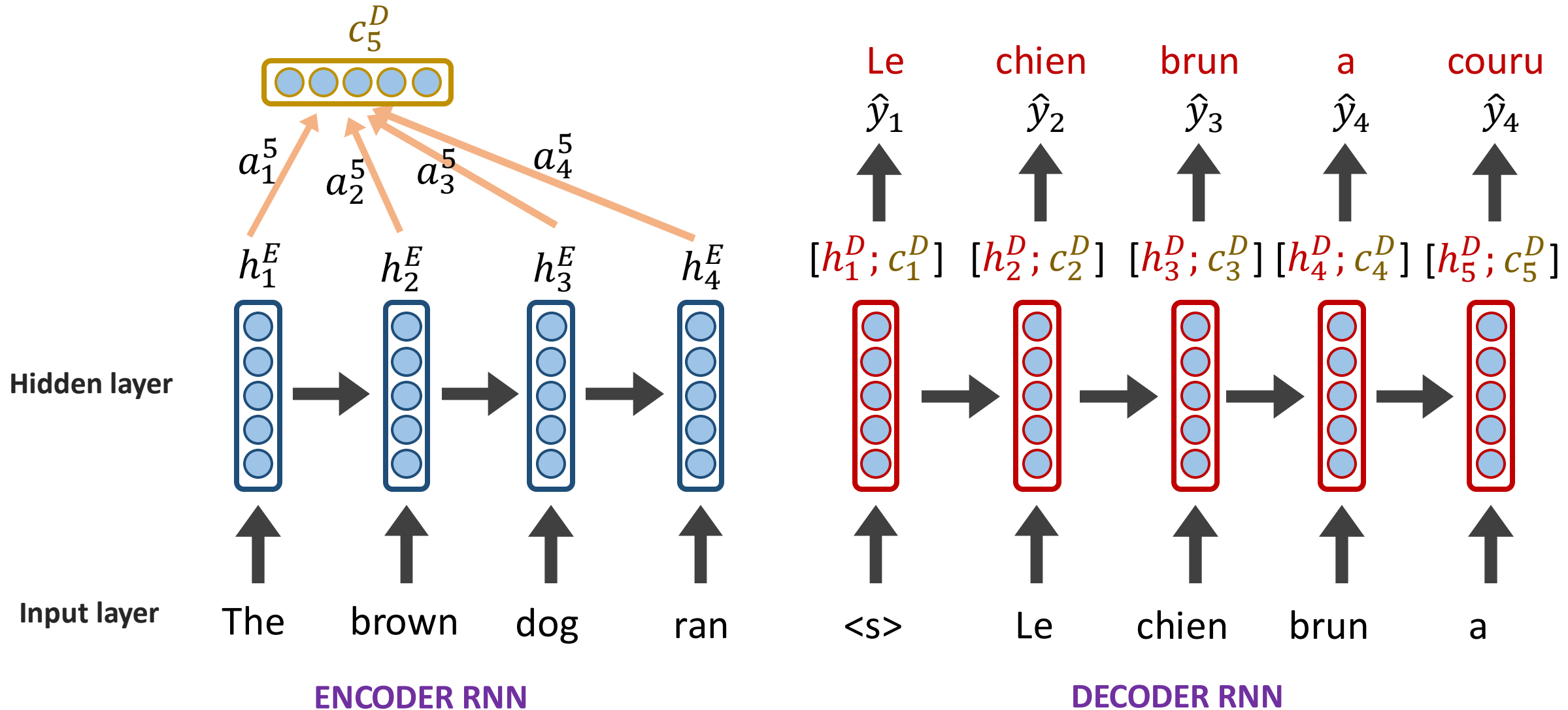
seq2seq + Attention

REMEMBER: each attention weight a_i^j is based on the **decoder's** current hidden state, too.

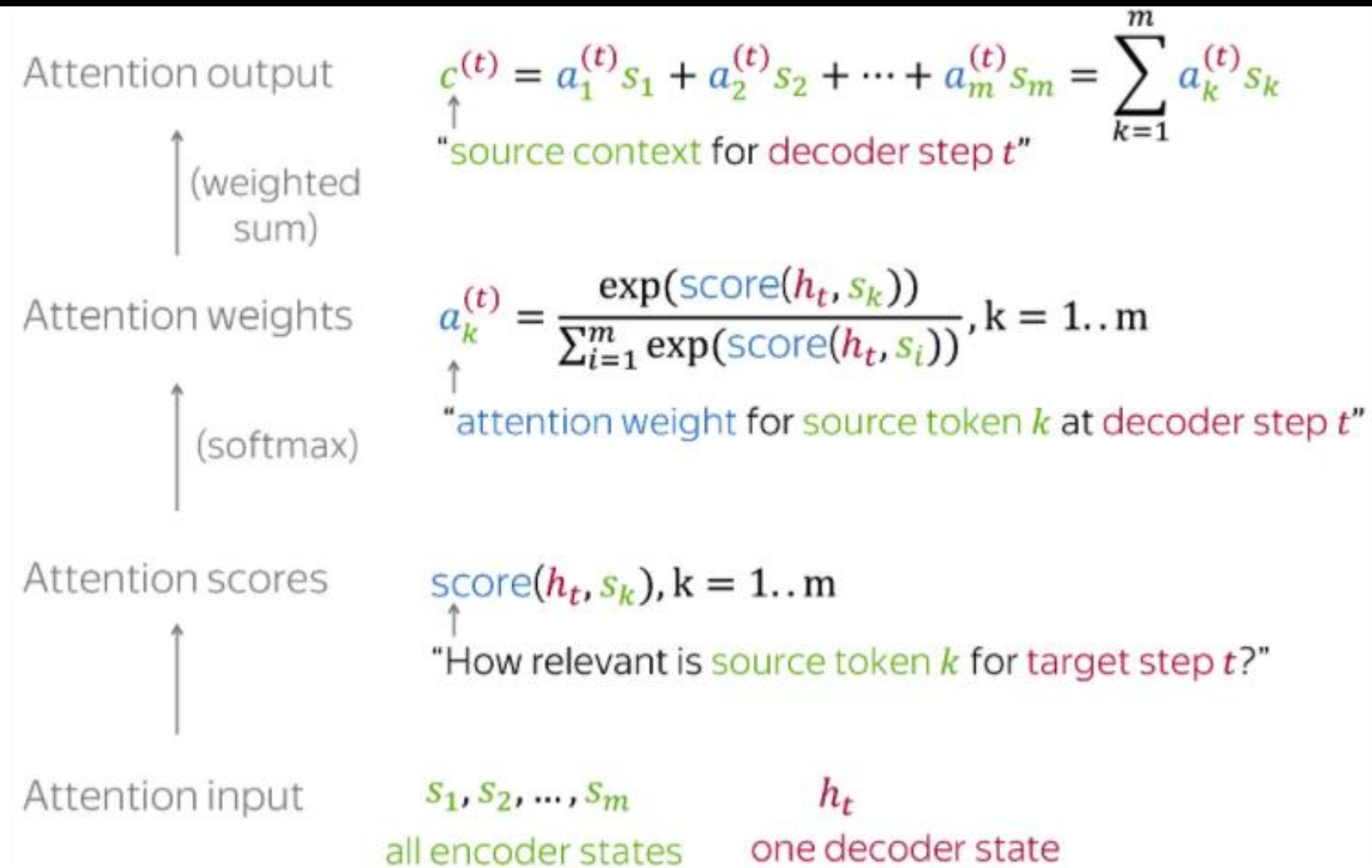


seq2seq + Attention

REMEMBER: each attention weight a_i^j is based on the **decoder's** current hidden state, too.



For convenience, here's the Attention calculation summarized on 1 slide

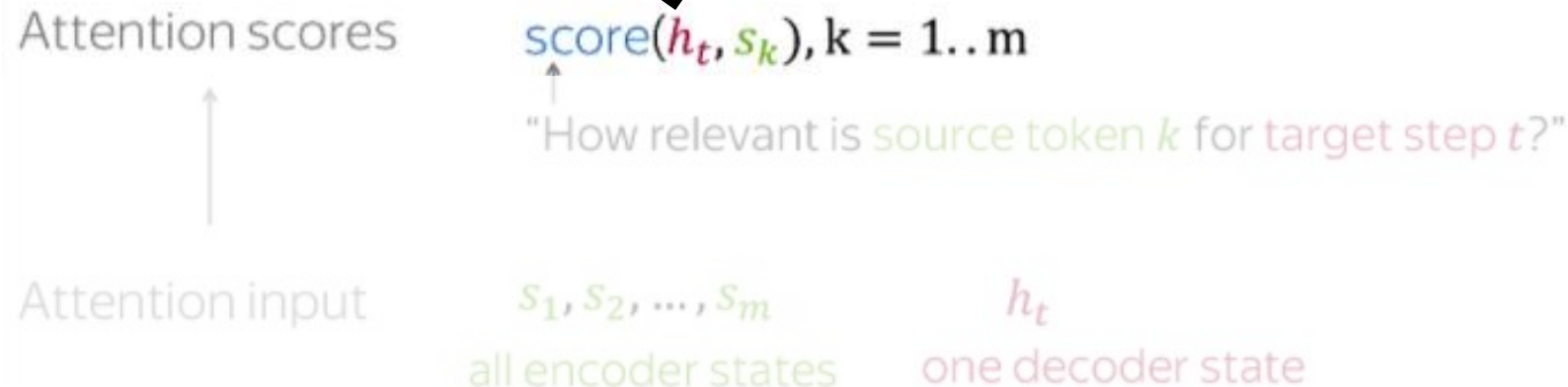


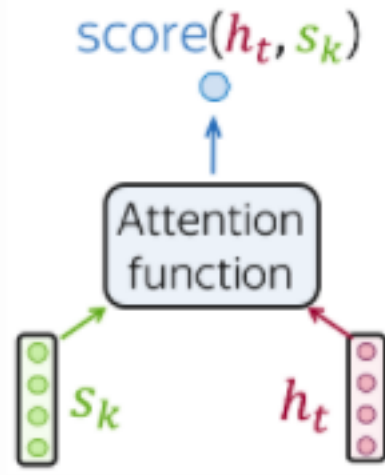
For convenience, here's the Attention calculation summarized on 1 slide

Attention output

$$c^{(t)} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \dots + a_m^{(t)} s_m = \sum_{k=1}^m a_k^{(t)} s_k$$

The **Attention mechanism** that produces scores doesn't have to be a **FFNN** like I illustrated. It can be any function you wish.





Popular Attention Scoring functions:

Dot-product

$$h_t^T \times s_k$$

$$\text{score}(h_t, s_k) = h_t^T s_k$$

Bilinear

$$h_t^T \times [W] \times s_k$$

$$\text{score}(h_t, s_k) = h_t^T W s_k$$

Multi-Layer Perceptron

$$w_2^T \times \tanh \left[W_1 \times \begin{bmatrix} h_t \\ s_k \end{bmatrix} \right]$$

$$\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1 [h_t, s_k])$$

seq2seq + Attention

Attention:

- greatly improves seq2seq results
- allows us to visualize the contribution each **encoding** word gave for each **decoder's** word

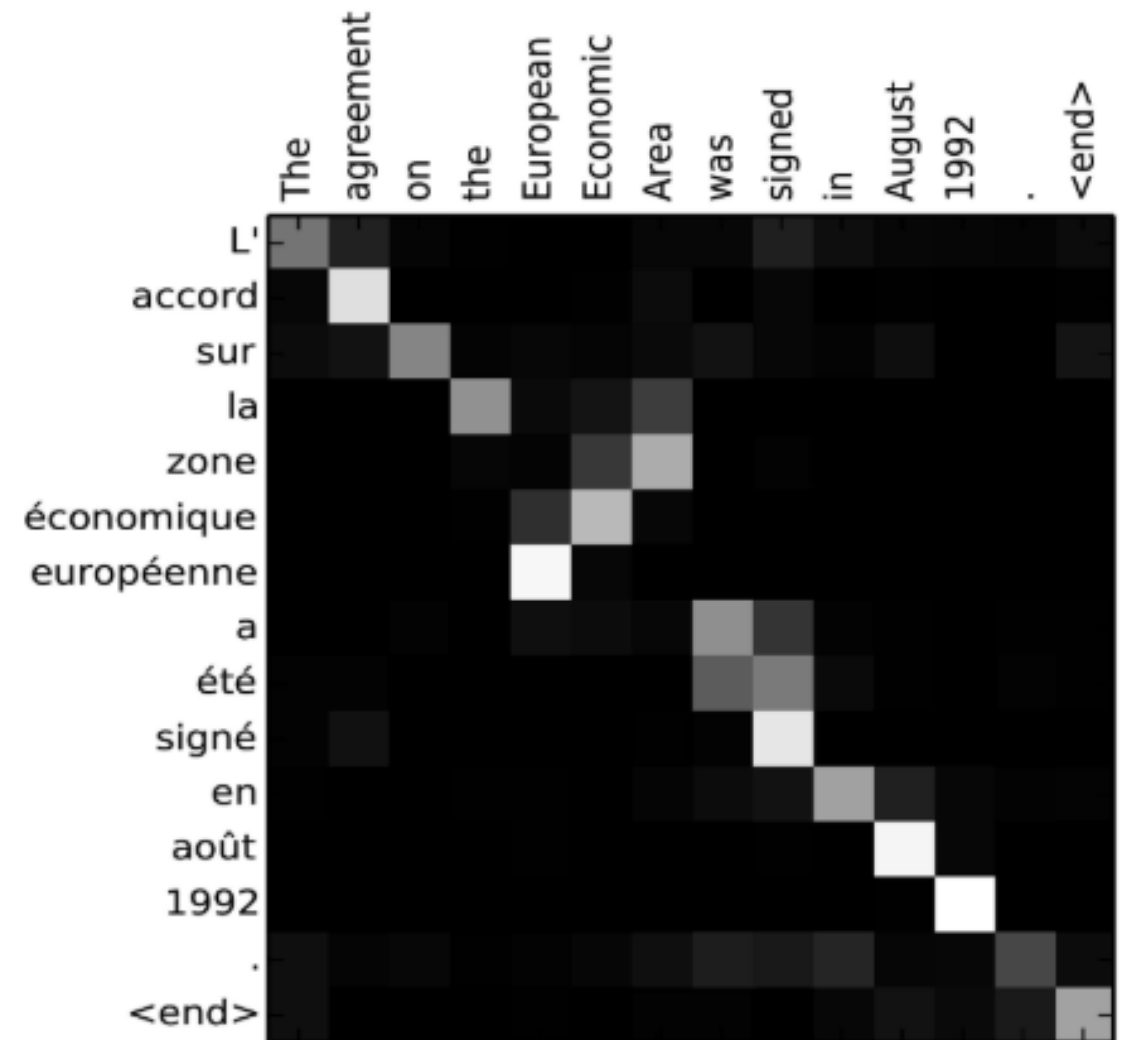
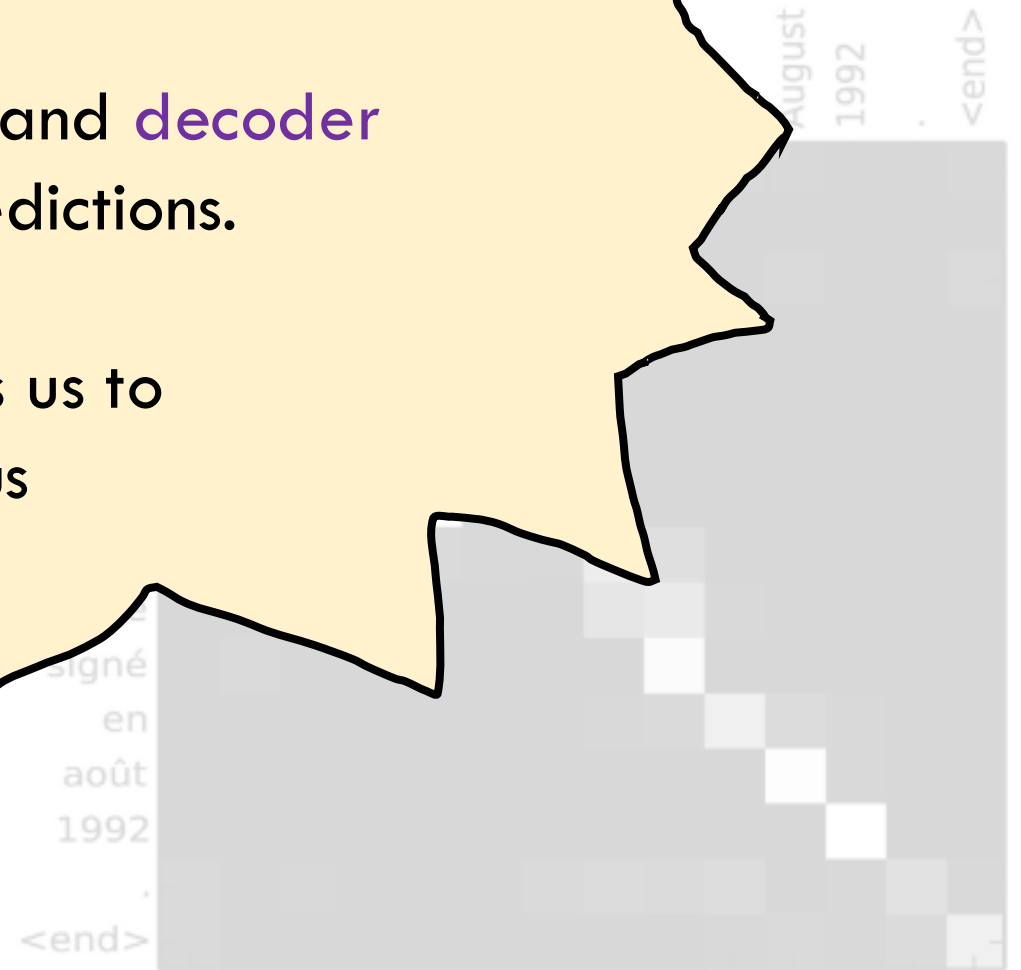


Image source: Fig 3 in [Bahdanau et al., 2015](#)

Takeaway:

Having a separate **encoder** and **decoder** allows for **n** $\rightarrow$ **m** length predictions.

Attention is powerful; allows us to conditionally weight our focus



SUMMARY

- **LSTMs** yielded state-of-the-art results on most NLP tasks (2014-2018)
- **seq2seq+Attention** was an even more revolutionary idea (Google Translate used it)
- **Attention** allows us to place appropriate weight to the encoder's hidden states
- But, **LSTMs** require us to iteratively scan each word and wait until we're at the end before we can do anything